

Laporan Milestone 1

Lexical Analysis

IF2224 Teori Bahasa Formal dan Automata



Kelompok JEE - SundalnJVM

Billie Bhaskara Wibawa	13524024
Raysha Erviandika Putra	13524050
Muhammad Naufal Romi Annafi	13524058
Dzaki Ahmad Al Hussainy	13524084

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

JL. GANESA 10, BANDUNG 40132

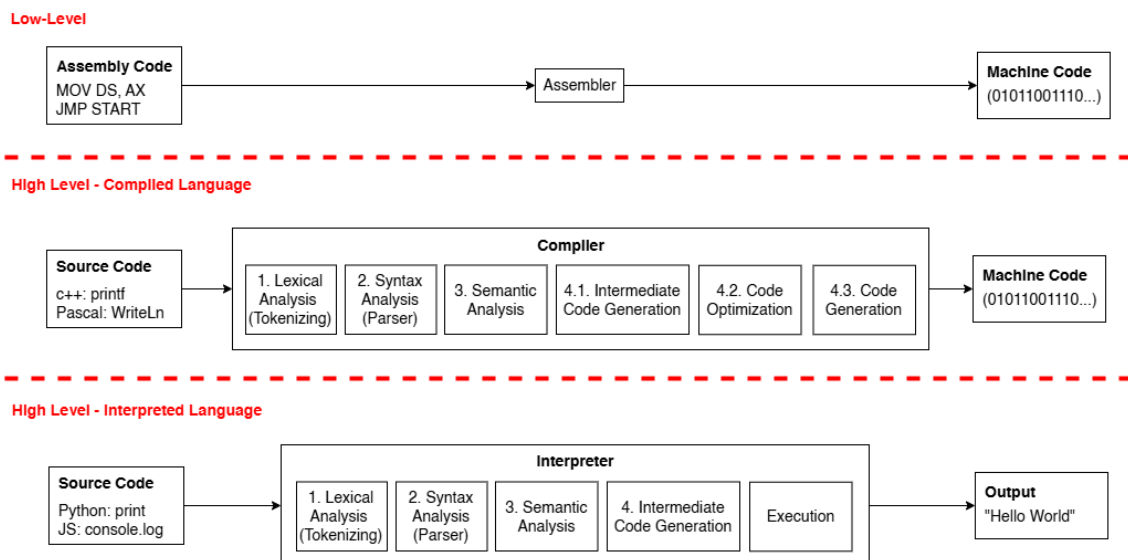
2026

Daftar Isi

Daftar Isi.....	2
1. Deskripsi Persoalan.....	3
2. Landasan Teori.....	5
2.1.1. Compiler/Interpreter, Kompilasi, dan Tahapannya.....	5
2.2. Lexical Analysis.....	5
2.3. Token, Lexeme, dan Pattern.....	6
2.4. Deterministic Finite Automata (DFA).....	7
2.5. Lexical Analysis dalam Arion Compiler.....	9
3. Perancangan dan Implementasi.....	11
3.1. Bahasa Pemrograman dan Design Pattern.....	11
3.2. Diagram DFA.....	11
3.2.1. Alpha Scanner.....	12
3.2.2. Numeric Scanner.....	16
3.2.3. Text Scanner.....	17
3.2.4. Symbol Scanner.....	18
3.3. Implementasi DFA.....	21
3.3.1. Global DFA.....	21
3.3.2. Alpha DFA (scanner Alpha).....	22
3.3.2.1. Overview.....	22
3.3.2.2. Definisi DFA.....	22
3.3.2.3. Implementasi Kode.....	25
3.3.3. Numeric DFA (scanner Numeric).....	26
3.3.3.1. Overview.....	26
3.3.3.2. Definisi transisi Numeric DFA.....	26
3.3.3.3. Implementasi Kode.....	27
3.3.4. Text DFA (scanner Text).....	30
3.3.4.1. Overview.....	30
3.3.4.2. Definisi transisi Text DFA.....	31
3.3.4.3. Implementasi Kode.....	31
3.3.5. Symbol DFA (scanner Symbol).....	34
3.3.5.1. Overview.....	34
3.3.5.2. Definisi DFA.....	34
3.3.5.3. Implementasi Kode.....	35
3.4. Alur Program.....	39
3.4.1. Inisialisasi LexicalAnalyzer.....	39
3.4.2. While Loop Inti.....	39
3.4.3. Pemaju Token.....	39
3.4.4. Terminasi.....	40
4. Pengujian.....	41
5. Penutup.....	55
5.1. Kesimpulan.....	55
5.2. Pesan.....	56
6. Lampiran.....	57

1. Deskripsi Persoalan

Bahasa pemrograman adalah cara agar kita dapat menuliskan program komputer tanpa perlu menghadapi sulitnya menulis dalam bahasa mesin. Bahasa mesin (biner 1 dan 0) adalah satu-satunya bahasa yang dapat dipahami dan dijalankan oleh CPU, namun sangat sulit dibaca oleh manusia. Oleh karena itu, bahasa pemrograman sering kali dibedakan dengan istilah low-level atau high-level. High-level artinya bahasa lebih mendekati bahasa manusia dan sebaliknya low-level lebih mendekati bahasa mesin. Semakin mendekati bahasa manusia, maka akan semakin banyak proses yang diperlukan untuk mengubahnya menjadi kode mesin.



Gambar 1.1 Tipe-tipe bahasa pemrograman

Terdapat dua macam cara mengubah/memproses bahasa pemrograman menjadi machine code, yaitu compiler dan interpreter. Compiler adalah program yang menerjemahkan seluruh source code menjadi kode mesin (dalam bentuk file e.g. Object File) sebelum program dijalankan. Compiler secara umum terdiri dari 4 tahapan, mulai dari Lexical Analysis, Syntax Analysis, Semantic Analysis hingga Intermediate Code Generation.

Di sisi lain adalah Interpreter yang memproses source code baris per baris secara langsung. Interpreter membaca, menerjemahkan, dan langsung mengeksekusi kode mesin baris per baris sehingga jika terdapat error pada baris ke 10, program akan tetap berjalan normal untuk 9 baris awal,

dibandingkan dengan compiler yang akan gagal dijalankan jika terdeteksi terdapat error di dalam source code. Namun dibalik itu, interpreter sebenarnya memiliki tahapan yang sama seperti compiler.

Pada Tugas Besar Teori Bahasa Formal dan Automata, tugasnya adalah diminta untuk membuat Interpreter untuk bahasa pemrograman Arion, suatu bahasa pemrograman baru yang akan dikembangkan. Tugas besar ini dibagi menjadi 4 Milestone, yaitu Lexical Analysis, Syntax Analysis, Semantic Analysis, Intermediate Code Generation dan Interpreter. Tugas besar kali ini membahas mengenai Lexical Analysis.

2. Landasan Teori

2.1.1. Compiler/Interpreter, Kompilasi, dan Tahapannya

Kompilasi adalah proses penerjemahan keseluruhan kode sumber yang ditulis dalam bahasa pemrograman tingkat tinggi seperti C++, Java, atau Pascal menjadi bahasa tingkat rendah atau bahasa mesin. Proses ini dilakukan secara menyeluruh dan sekaligus sebelum program tersebut dijalankan. Hasil dari proses kompilasi biasanya berupa file executable misalnya .exe di Windows yang dapat langsung dipahami dan dieksekusi oleh perangkat keras komputer.

Compiler adalah alat untuk melakukan kompilasi yaitu alat yang dapat membaca suatu bahasa pemrograman dari kode sumber dan mentranslasikannya menjadi bahasa pemrograman target. Salah satu fungsi compiler adalah untuk mendeteksi adanya kesalahan atau error yang dapat terdeteksi dalam kode sumber ketika proses translasi.

Sedangkan, interpreter adalah perangkat lunak yang menerjemahkan dan mengeksekusi kode sumber baris demi baris secara langsung pada saat program dijalankan. Berbeda dengan compiler yang memproses seluruh kode di awal untuk menghasilkan file executable, interpreter mengeksekusi instruksi secara real-time dan akan langsung berhenti serta menampilkan pesan error jika menemukan kesalahan pada baris yang sedang diproses.

Dalam arsitektur sebuah interpreter, proses penerjemahan yang kompleks ini dibagi menjadi beberapa fase:

1. Lexical Analysis: Memecah karakter menjadi token
2. Syntax Analysis: Memeriksa struktur gramatikal
3. Semantic Analysis: Memeriksa konsistensi makna
4. Intermediate Code Generation: Membuat kode intermediate

Pada Milestone 1 Tugas Besar ini, fokus implementasi berada pada tahap pertama yaitu Lexical Analysis yang merupakan proses pertama dari proses kompilasi bahasa pemrograman.

2.2. Lexical Analysis

Lexical Analysis atau kadang disebut sebagai tahapan scanning adalah fase pertama dalam proses kompilasi. Pada tahap ini, interpreter membaca kode

sumber mentah secara sekuensial yaitu karakter demi karakter dari awal hingga akhir dan mengelompokkan rangkaian karakter tersebut menjadi kesatuan-kesatuan dasar yang memiliki makna spesifik yang disebut sebagai token.

Selain bertugas untuk menghasilkan barisan token, Lexical Analyzer juga menjalankan beberapa fungsi penting lainnya seperti: Menyaring karakter yang tidak relevan seperti spasi, tab, newline; Melacak posisi kode karena Lexer menyimpan informasi mengenai nomor baris dan kolom dari karakter yang sedang dibaca. Hal ini sangat krusial untuk menghasilkan pesan error yang presisi sehingga memudahkan proses debugging; Mendeteksi kesalahan leksikal yaitu jika Lexer menemukan karakter atau simbol asing yang tidak dikenali dan tidak cocok dengan pola bahasa pemrograman yang telah didefinisikan, program akan mencatatnya sebagai lexical error atau unrecognized token.

Keluaran utama dari tahap Lexical Analysis ini adalah aliran token. Aliran token inilah yang kemudian akan dikirim ke fase kompilasi berikutnya, yaitu Syntax Analysis (atau Parser), untuk diperiksa apakah susunan token-token tersebut sudah membentuk struktur tata bahasa sintaks yang benar.

2.3. Token, Lexeme, dan Pattern

Token merupakan unit terkecil yang memiliki arti dalam sebuah program. Token adalah simbol abstrak atau label kategori yang diberikan oleh Lexer untuk merepresentasikan jenis dari suatu kata atau elemen dasar. Token inilah yang nantinya akan dikirim ke tahapan Syntax Analysis. Contoh token misalnya kata kunci (keyword), nama variabel (identifier), operator, angka, atau tanda baca.

Ada istilah lain juga dalam proses Lexical Analysis yaitu Lexeme dan Pattern. Lexeme adalah wujud nyata dari token yang merupakan rangkaian karakter aktual (teks asli) di dalam source code yang diketikkan oleh pemrogram, yang memenuhi aturan dari suatu token tertentu. Sedangkan, Pattern adalah aturan baku yang mendeskripsikan ciri-ciri atau struktur karakter yang harus dipenuhi oleh sebuah lexeme agar bisa diklasifikasikan ke dalam suatu kategori token. Pola ini bertindak sebagai kriteria pengujian bagi Lexer.

Misalkan, dalam contoh potongan kode C sederhana: `int nilai = 100;`

Proses pemecahannya dapat diilustrasikan sebagai berikut:

Token	Lexeme	Pattern
int	KEYWORD_INT	Rangkaian karakter pasti 'i', 'n', 't'
nilai	IDENTIFIER	Diawali huruf, diikuti oleh huruf atau angka
=	ASSIGN_OP	Karakter tunggal '='
100	NUMBER	Rangkaian digit angka (0-9)
;	SEMICOLON	Karakter tunggal ';'

Melalui contoh di atas, dapat terlihat bahwa dalam proses Lexical Analysis, Lexer bekerja dengan memindai lexeme (teks asli) satu per satu, lalu mencocokkannya dengan pattern (pola) yang ada, terakhir membungkusnya menjadi token (unit kategori).

2.4. Deterministic Finite Automata (DFA)

Finite Automata (FA) adalah model matematika abstrak berupa mesin dengan sejumlah keadaan atau state yang digunakan untuk mengenali suatu bahasa atau pola tertentu. Dalam perancangan compiler, jenis FA yang secara khusus digunakan untuk mengimplementasikan Lexer adalah Deterministic Finite Automata (DFA).

DFA disebut deterministik karena mesin ini memiliki kepastian arah. Artinya, pada setiap state dan untuk setiap simbol karakter input yang dibaca, hanya terdapat tepat satu transisi atau perpindahan ke state berikutnya. DFA tidak mengizinkan adanya ambiguitas ataupun transisi kosong (epsilon transition).

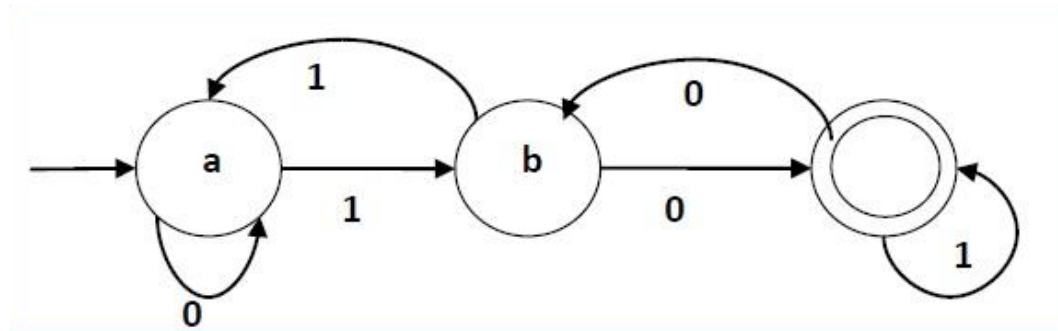
Secara formal, DFA didefinisikan sebagai sistem 5 tuple yaitu

$$M = (Q, \Sigma, \delta, q_0, F)$$

Dalam hal ini

- Q adalah himpunan state yang terbatas
- Σ adalah himpunan semua simbol atau karakter input yang valid
- $\delta: Q \times \Sigma \rightarrow Q$ adalah fungsi transisi
- $q_0 \in Q$ adalah state awal

- $F \subseteq Q$ adalah himpunan state akhir



Gambar 2.1 Contoh Diagram DFA

Dalam implementasi Lexer, digunakan prinsip maximal munch atau longest match. Lexer selalu mencoba membentuk token terpanjang yang mungkin dikenali. Algoritma ini bekerja dengan:

1. Mulai dari start state
2. Baca karakter dan lakukan transisi
3. Catat posisi terakhir saat berada di final state
4. Lanjutkan membaca sampai tidak ada transisi valid
5. Rollback ke posisi final state terakhir
6. Emit token dan ulangi dari start state

Sering dalam implementasi Lexer digunakan Dead state. Dead state adalah sebuah state perangkap (trap state). Jika DFA masuk ke state ini, artinya kombinasi karakter yang dibaca sudah pasti salah sejak awal dan tidak ada satu pun jalur transisi yang bisa membawanya menuju accepting state. Penggunaan dead state membuat Lexer bekerja lebih efisien karena bisa langsung menolak karakter ilegal tanpa harus mengecek keseluruhan input.

Selain itu, Jika mesin DFA berhenti karena bertemu karakter spasi, karakter tidak dikenali, atau masuk ke dead state, namun tidak berada di accepting state, maka Lexer akan memicu status Lexical Error. Pada situasi ini, Lexer akan mencatat pesan error tersebut beserta posisi baris dan kolomnya untuk keperluan debugging, kemudian melakukan error recovery dengan membuang karakter yang bermasalah dan mereset sistem kembali ke start state guna melanjutkan pencarian token berikutnya.

2.5. Lexical Analysis dalam Arion Compiler

Dalam Milestone 1 ini, dibuat sebuah Lexical Analyzer yang berfungsi sebagai tahap pertama dalam pipeline kompilasi untuk bahasa pemrograman Arion. Lexer ini diimplementasikan menggunakan Deterministic Finite Automata (DFA).

Mesin DFA di dalam Lexer diprogram secara spesifik untuk mengenali dan mengklasifikasikan 52 jenis token dasar yang mendefinisikan bahasa Arion. Klasifikasi ini mencakup:

- Keywords

Keyword khusus bahasa yang tidak dapat digunakan sebagai variabel seperti `programsy`, `varsy`, `beginsy`, `ifsy`.

- Identifiers

Nama variabel atau fungsi yang diawali huruf dan diikuti kombinasi huruf atau angka. Sesuai aturan Arion, pembacaan identifier ini bersifat case-insensitive atau tidak membedakan huruf besar dan kecil.

- Konstanta

Meliputi angka bulat (`intcon`), angka desimal/riil (`realcon`), karakter tunggal (`charcon`), serta untaian teks (`string`).

- Operator dan Simbol

Meliputi operator aritmatika, operator logika, operator perbandingan, hingga simbol tanda baca seperti titik, koma, titik dua.

Selain mengumpulkan lexeme yang bermakna, Lexer juga dirancang untuk membuang karakter yang tidak dieksekusi. Karakter whitespace seperti spasi, tab, newline akan diabaikan. Selain itu, program juga memiliki state khusus pada DFA untuk menangani dan membuang blok komentar bahasa Arion, baik yang menggunakan format kurung kurawal `{...}` maupun format kurung-bintang `(* ... *)`.

Implementasi DFA untuk lexer ini memiliki beberapa tantangan tersendiri. Pertama, kata kunci (keyword) seperti `begin` dan `end` memiliki struktur leksikal yang identik dengan identifier (ident) biasa, yakni diawali dengan huruf. Oleh karena itu, Lexer memerlukan mekanisme pengecekan string

lanjutan sesaat setelah pola identifier berhasil dikumpulkan untuk membedakan statusnya.

Kedua, kehadiran operator multi-karakter seperti `:=` (becomes), `<=` (leq), `>=` (geq), dan `<>` (neq) mengharuskan implementasi mekanisme lookahead. Karakter awalan dari operator-operator ini tidak bisa langsung disimpulkan karena bisa berdiri sendiri; sebagai contoh, karakter `:` bisa menjadi token tunggal colon atau menjadi bagian dari operator penugasan `:=`. Mekanisme lookahead yang sama juga krusial untuk membedakan kurung buka biasa `(` (lparent) dengan awalan blok komentar `(*`.

Ketiga, pembacaan bilangan yang memiliki nilai desimal seperti 3.14 harus dikenali secara utuh sebagai satu kesatuan token realcon, bukan dipecah menjadi token intcon dan karakter lainnya. Hal ini sangat berkaitan dengan tantangan terakhir, yakni ambiguitas karakter titik `.`. Dalam bahasa Arion, karakter titik memiliki dua kemungkinan interpretasi utama: sebagai pemisah desimal pada bilangan riil (3.14), atau sebagai penanda akhir program/token period tunggal (seperti pada end.). Kondisi ini menuntut rancangan perpindahan state yang sangat presisi agar mesin DFA tidak salah dalam mengambil keputusan.

3. Perancangan dan Implementasi

3.1. Bahasa Pemrograman dan Design Pattern

Implementasi Lexical Analyzer ini dibangun menggunakan bahasa pemrograman C++ dengan beberapa pertimbangan teknis utama. Pemilihan C++ didasarkan pada keunggulannya dalam performa eksekusi dan efisiensi saat memproses karakter demi karakter dari source code. Selain itu, pemanfaatan Standard Template Library (STL) sangat membantu proses pengembangan.

Secara arsitektur, program dirancang secara modular dengan memadukan paradigma Object-Oriented Programming (OOP) dan pemisahan namespace agar tanggung jawab setiap komponen terdefinisi dengan jelas. Terdapat komponen Token yang bertugas merepresentasikan entitas hasil pemindaian beserta informasi posisinya (baris dan kolom). Berbeda dengan sistem yang membaca aturan transisi secara eksternal, logika pemrosesan Deterministic Finite Automata (DFA) pada sistem ini diimplementasikan langsung secara terpisah ke dalam namespace SpecificScanners (seperti pemindai alfanumerik, numerik, dan simbol). Selanjutnya, kelas LexicalAnalyzer bertindak sebagai dispatcher utama yang memegang kendali atas kursor pembacaan melalui struct LexerState, mengarahkan aliran karakter ke scanner yang tepat, dan menangani keseluruhan alur pembentukan token dari awal hingga akhir.

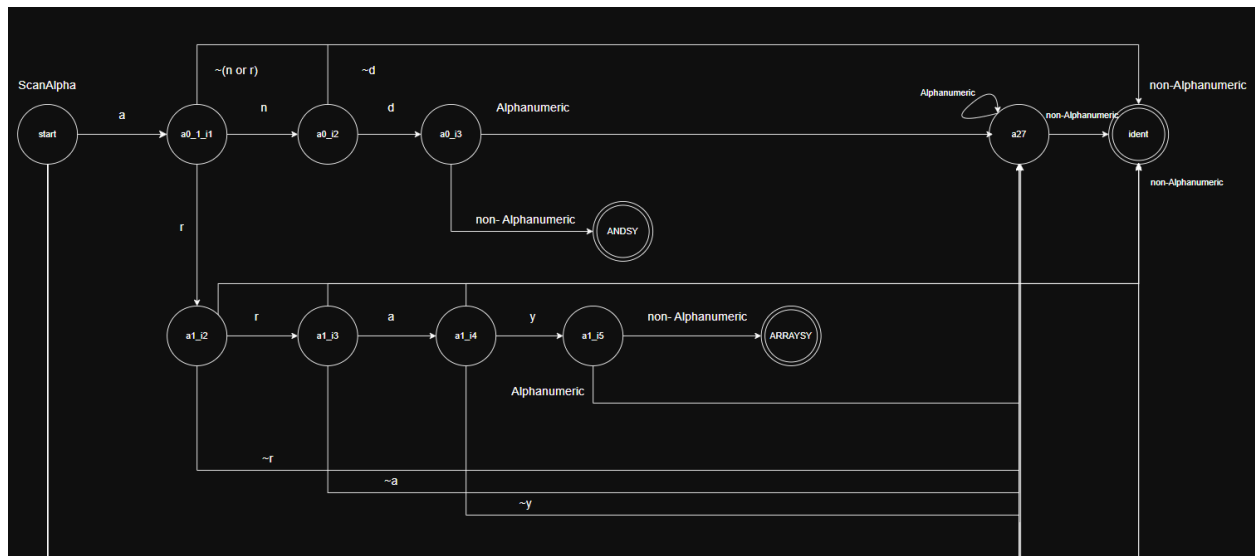
3.2. Diagram DFA

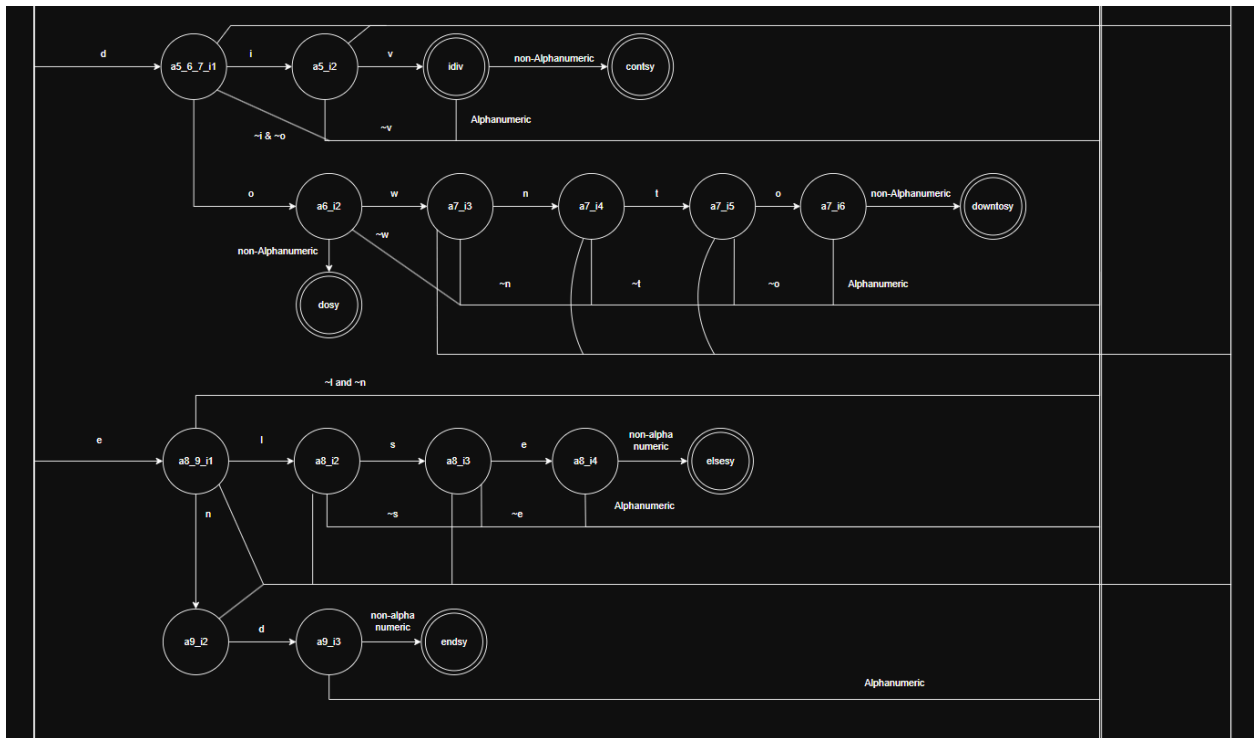
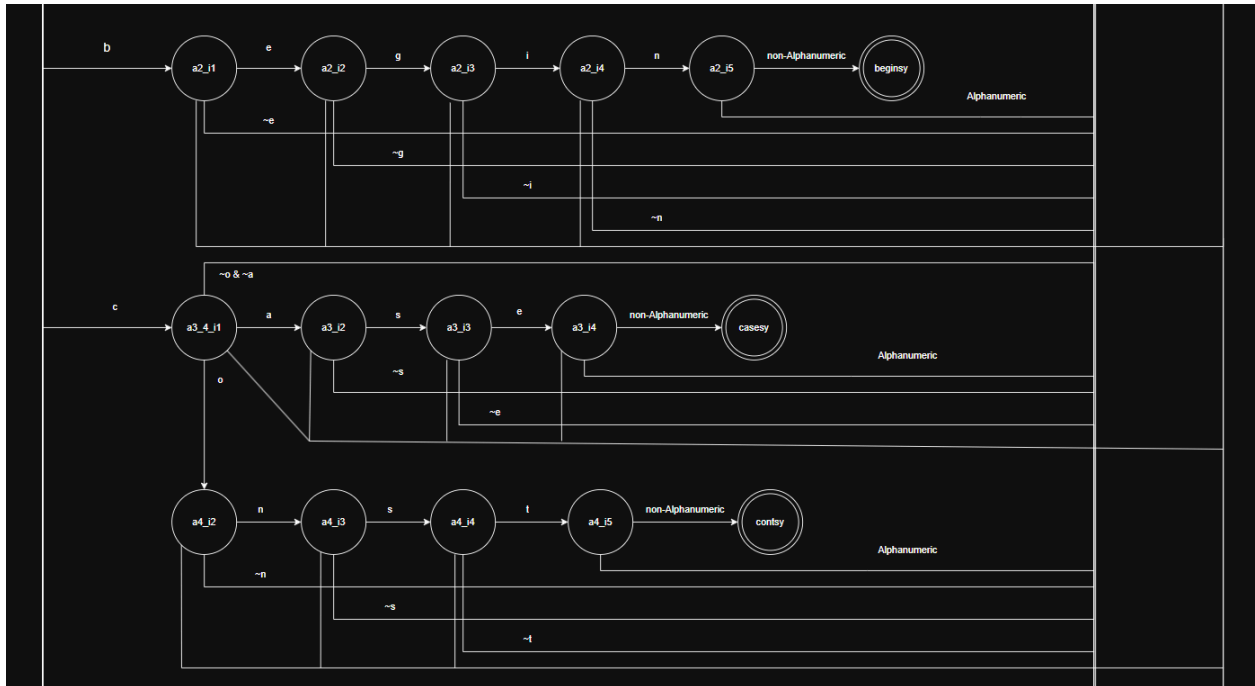
Di bawah ini merupakan diagram dari DFA Lexical Analyzer yang akan diimplementasikan. Pembuatan diagram DFA diabstraksi menjadi beberapa bagian yaitu AlphaScanner untuk translasi keywords dan variabel, TextScanner untuk translasi string, NumericScanner untuk translasi konstanta angka, dan SymbolScanner untuk translasi simbol (operator matematika, tanda baca, tanda kurung). Alasan DFA ini dibagi menjadi beberapa bagian karena pembuatan state yang sebenarnya akan dirasa terlalu banyak. Misalkan, pada AlphaScanner, proses translasi pada setiap keywords berakhir dengan non-Alphanumeric. Lexeme non-Alphanumeric akan memulai salah satu dari SymbolScanner atau TextScanner. Jika dibuat implementasi aslinya, hal ini membutuhkan setiap final state dari AlphaScanner mempunyai pasangan seluruh state awal dari SymbolScanner

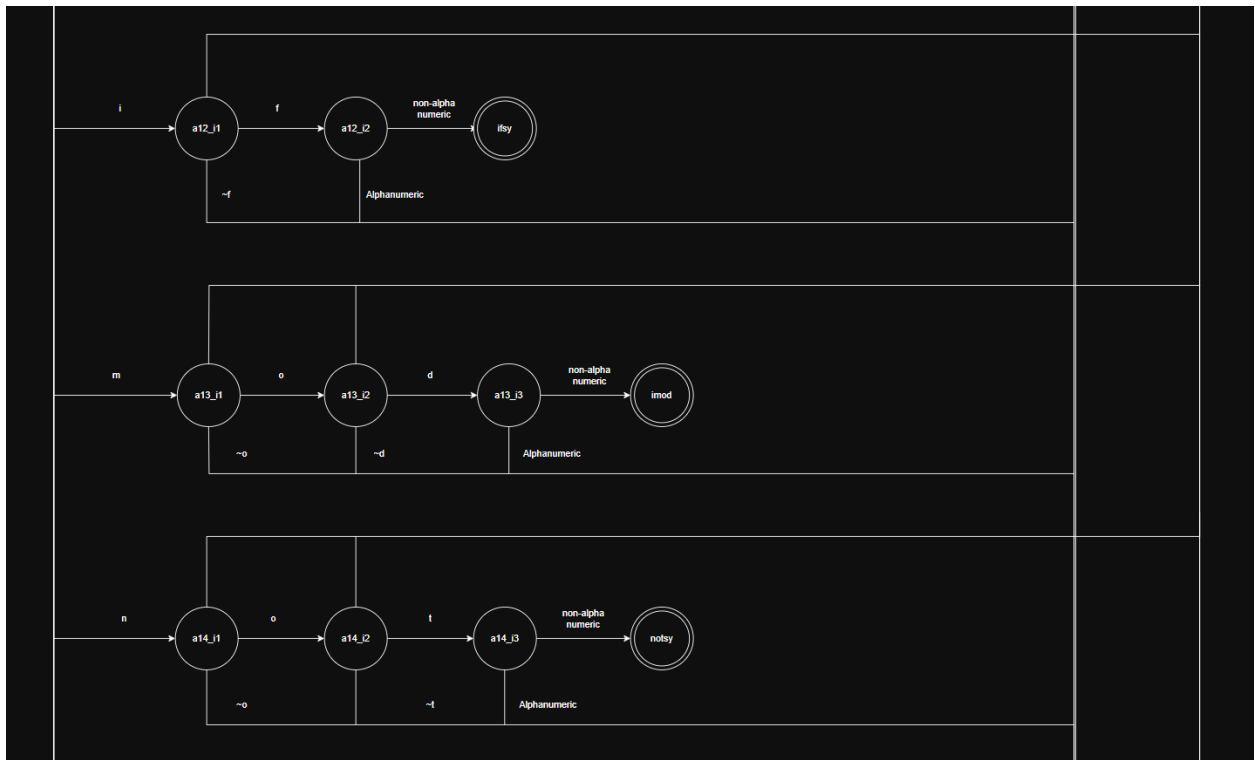
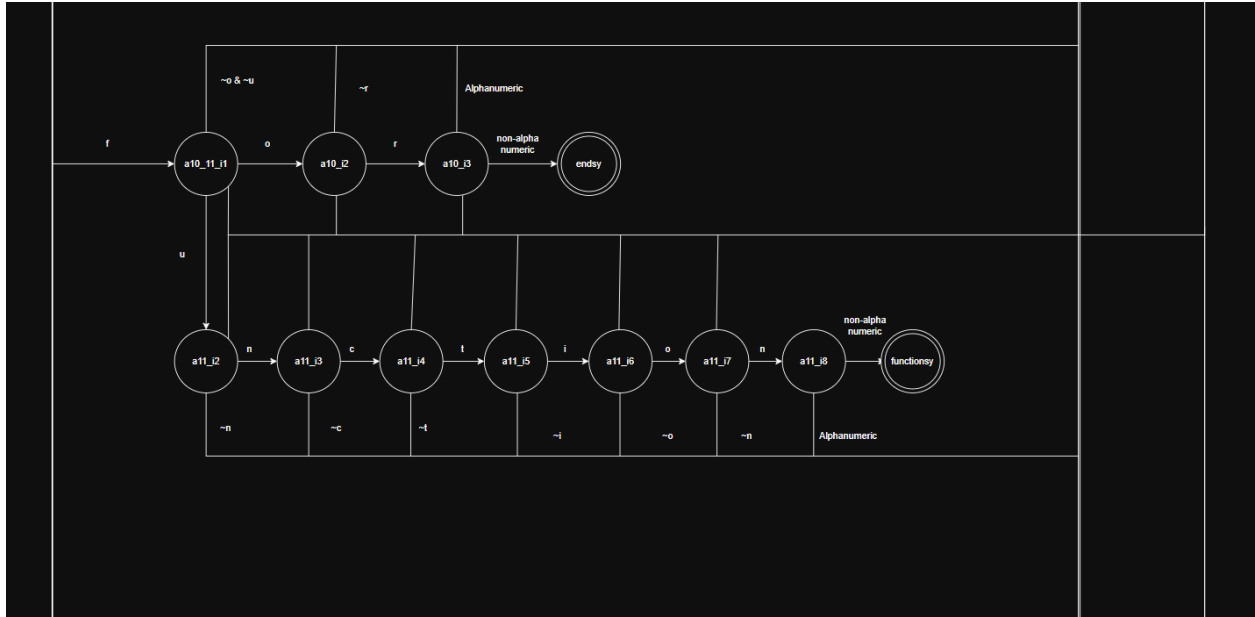
dan TextScanner. Jumlah final state AlphaScanner ada 27 dengan Jumlah state awal SymbolScanner ada 26 dan jumlah state awal TextScanner ada 1 menjadi final State dapat menjadi sebanyak $27 \times (26+1) = 756$ final state. Hal ini menjadi pembuatan diagram akan sangat sulit sehingga digunakan abstraksi diagram DFA ini.

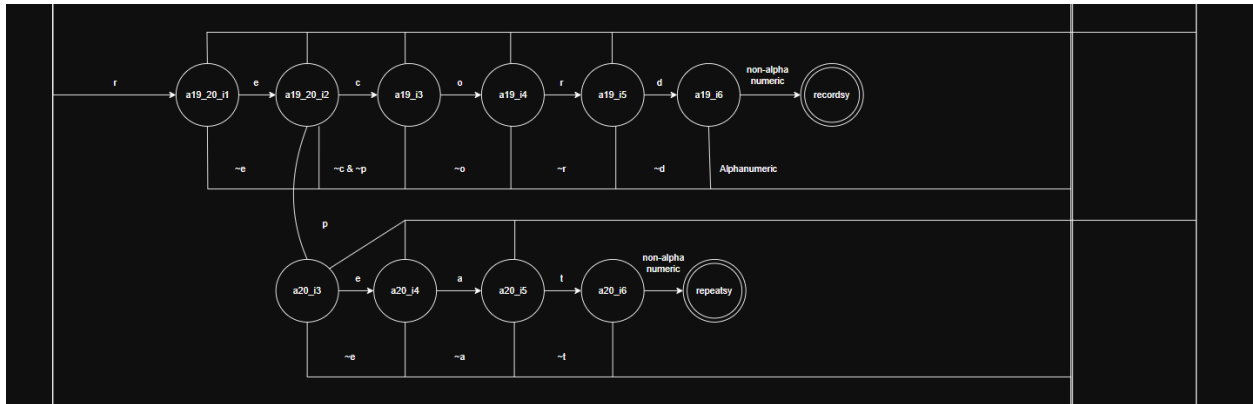
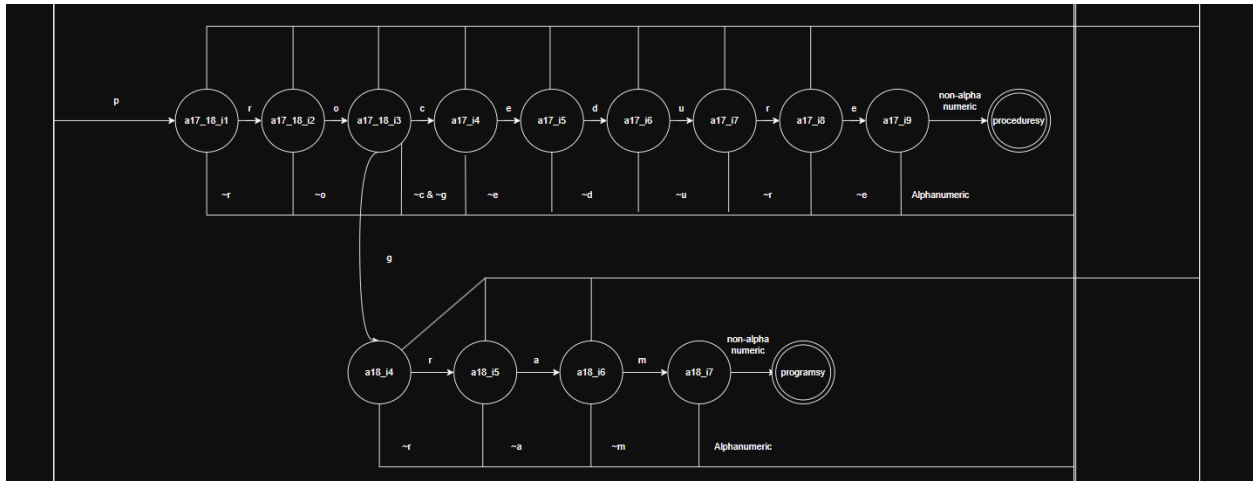
3.2.1. Alpha Scanner

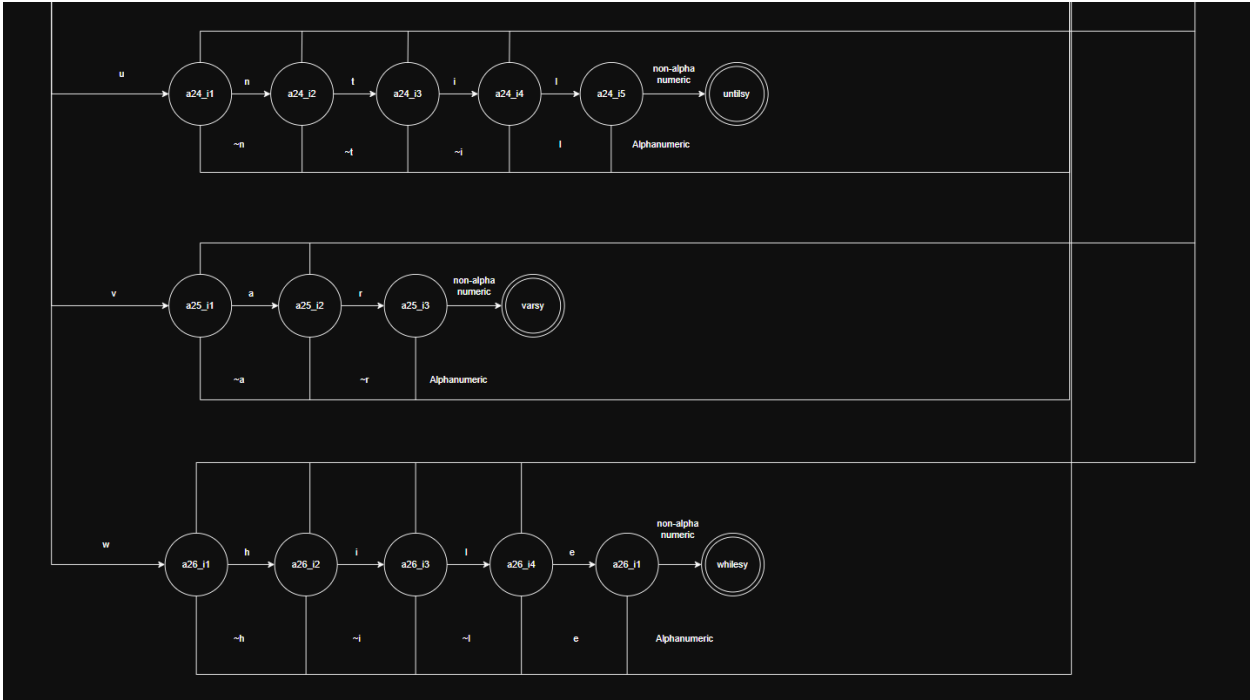
Dalam AlphaScanner ini maksud ~n adalah scope-nya AlphaNumeric bukan semua karakter, melainkan semua Alphanumeric selain n dengan n suatu alfabet huruf



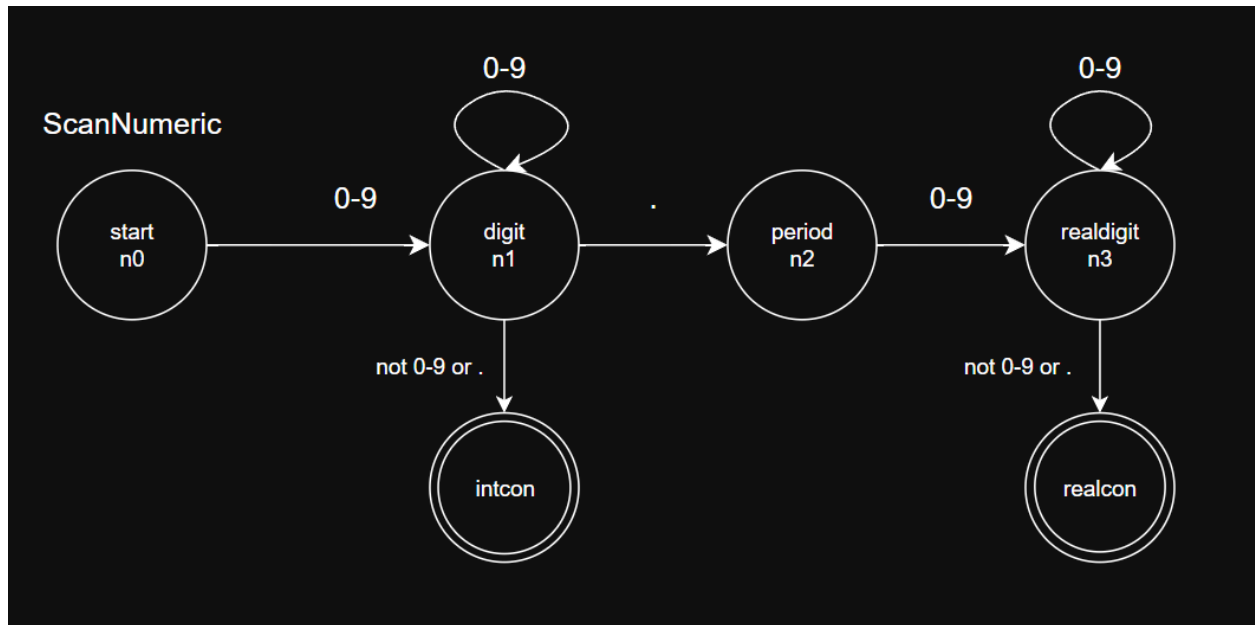




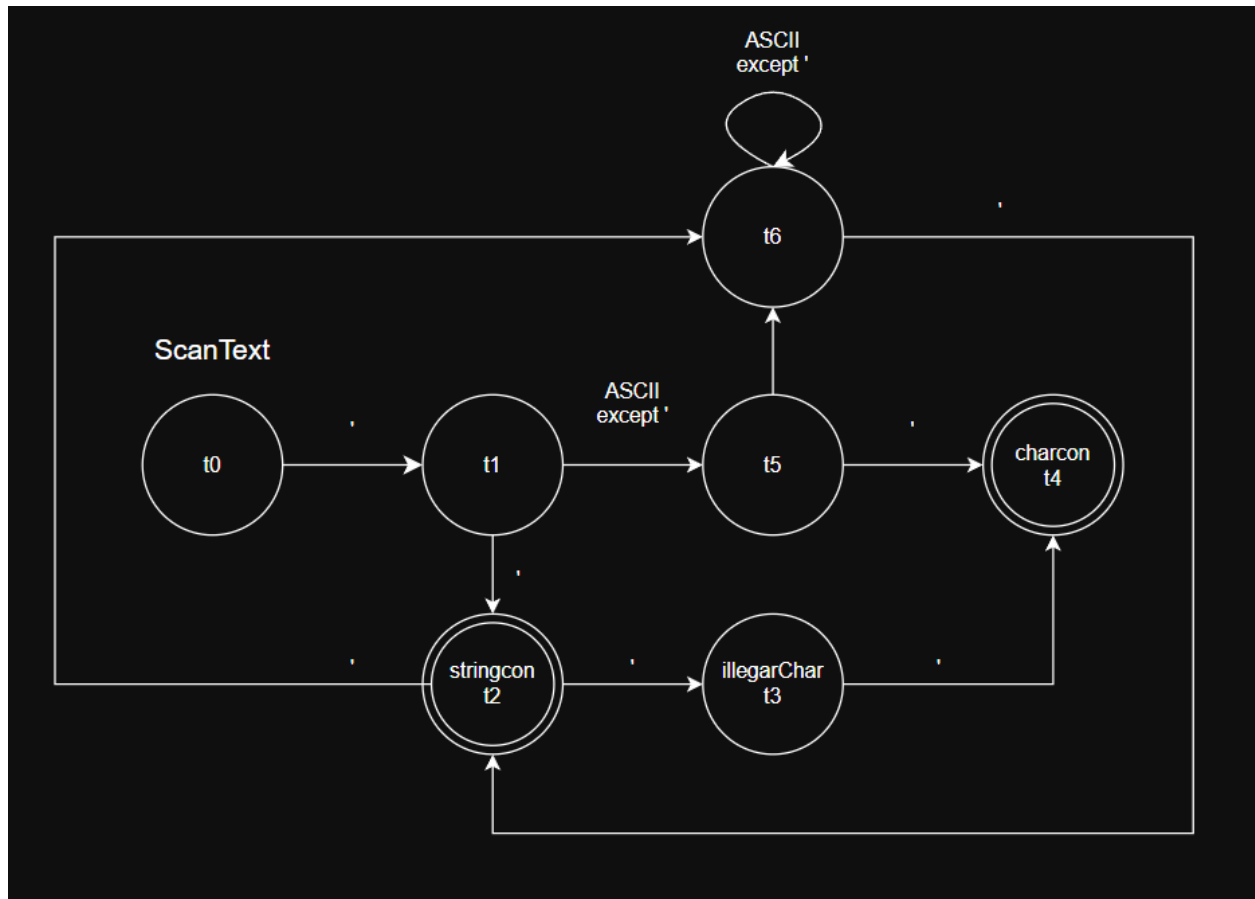




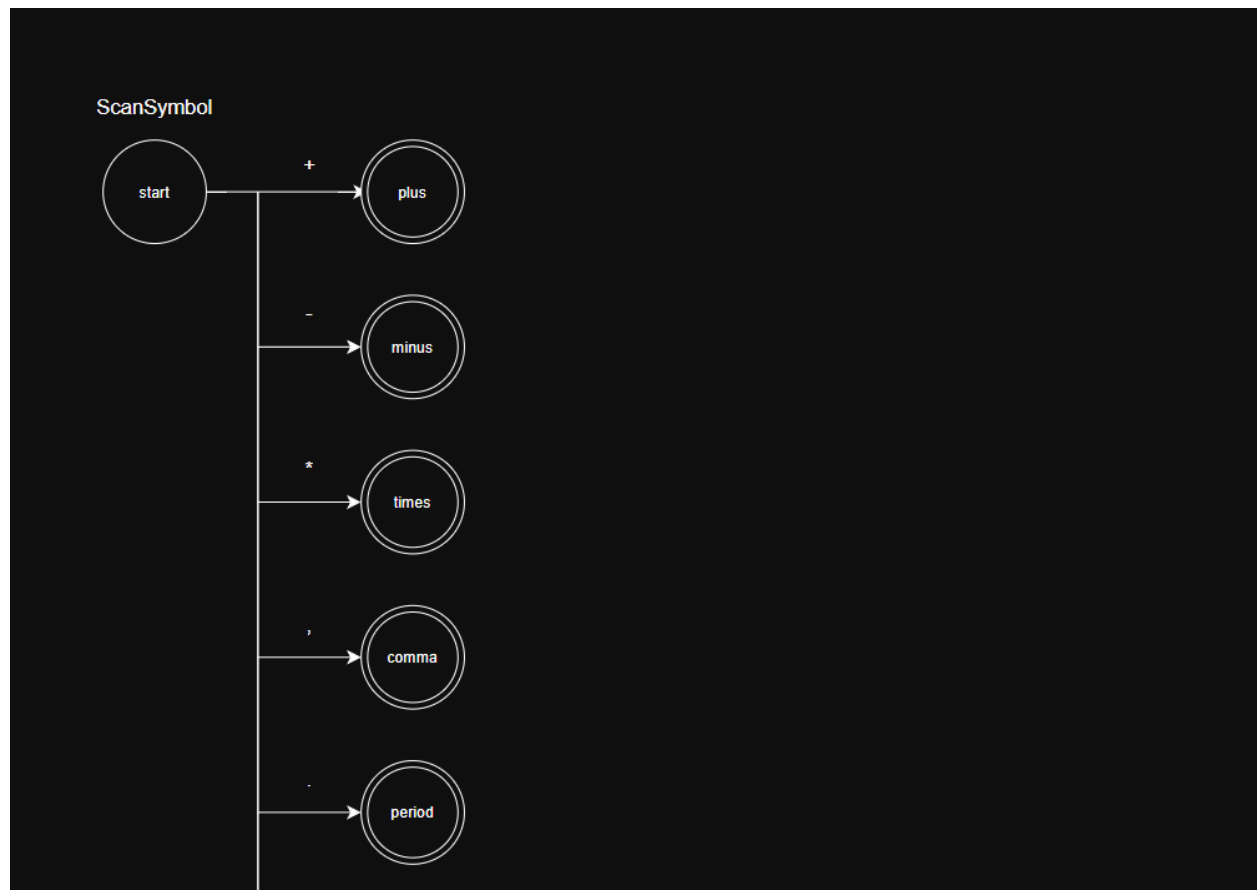
3.2.2. Numeric Scanner

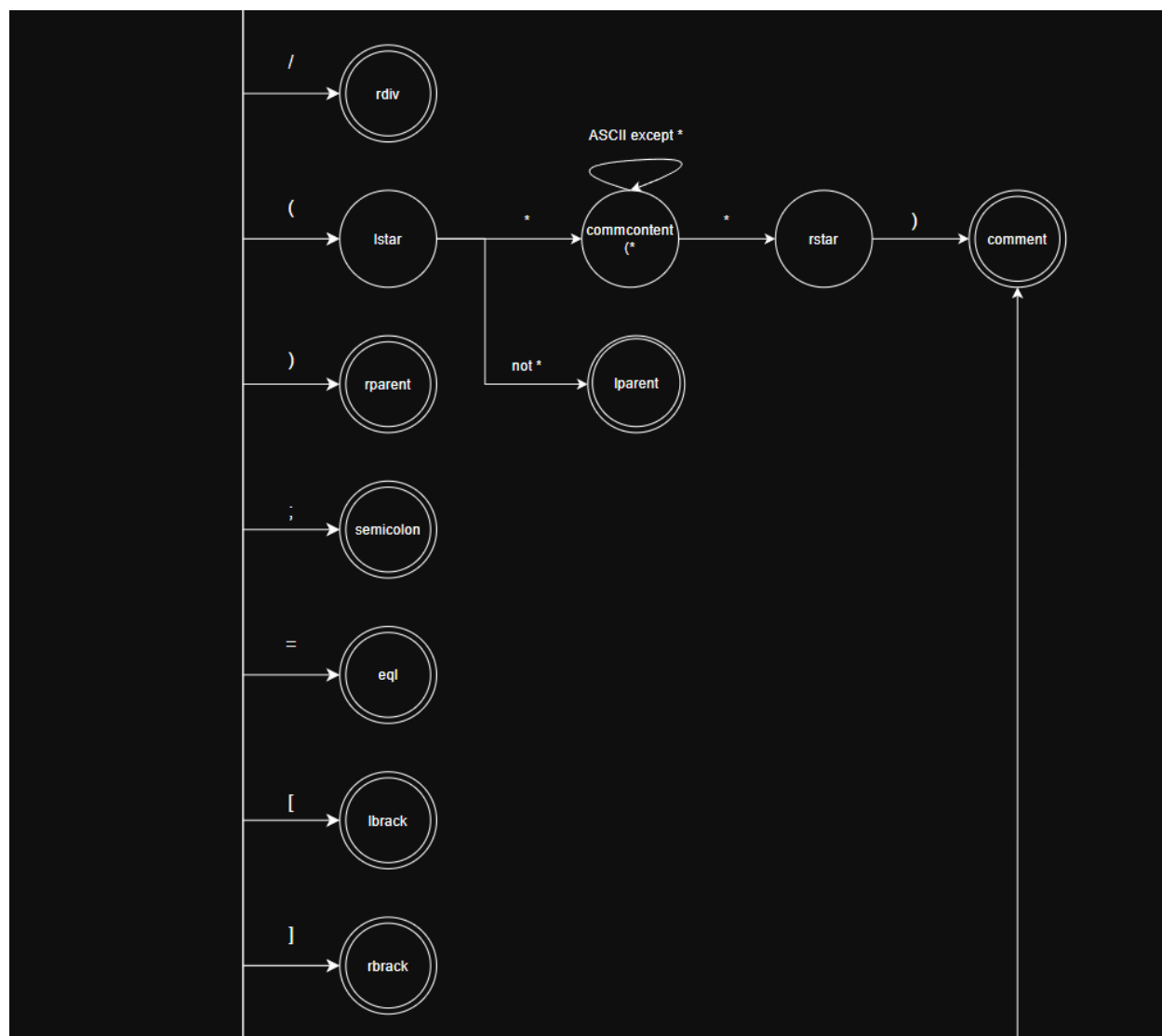


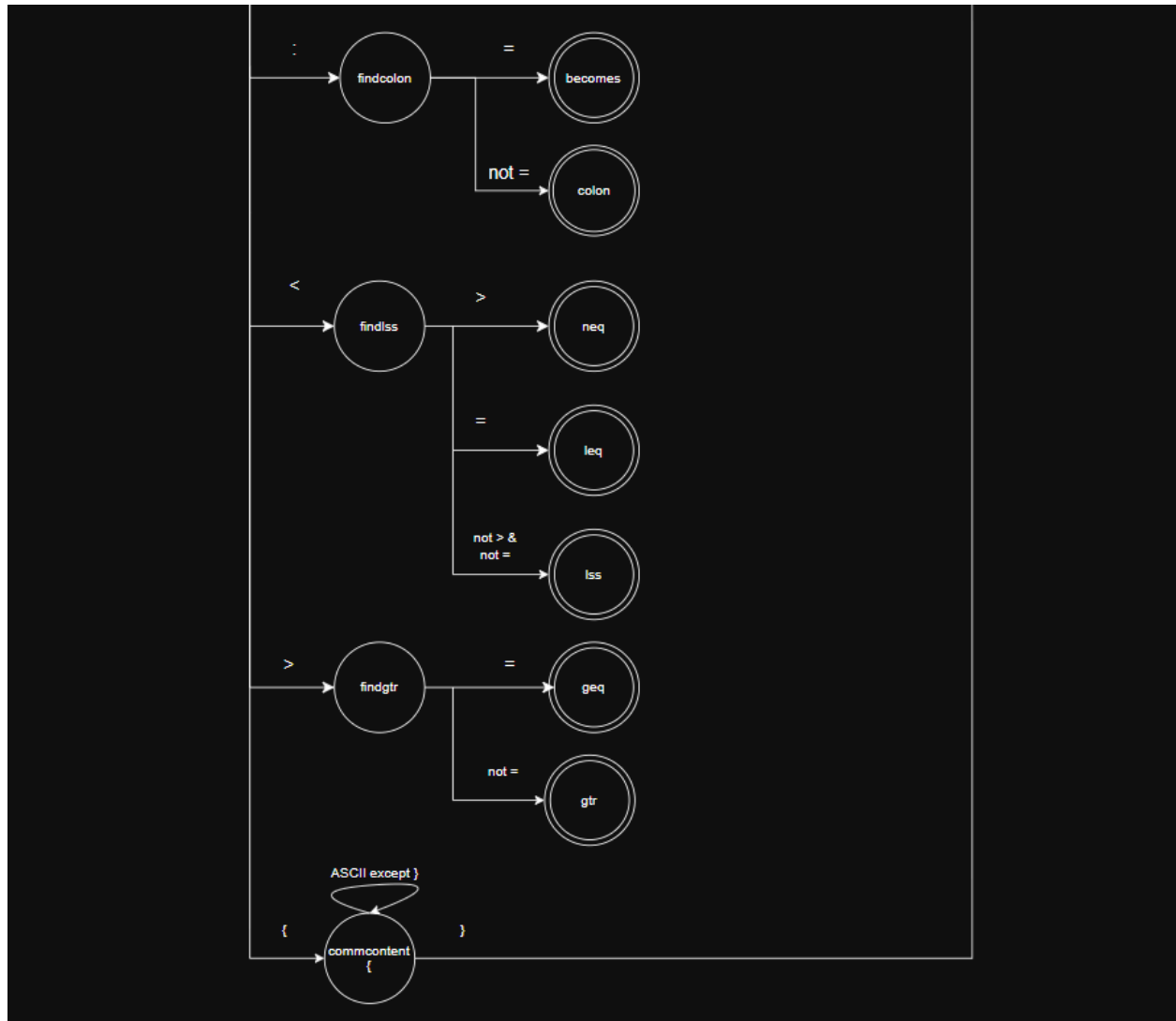
3.2.3. Text Scanner



3.2.4. Symbol Scanner

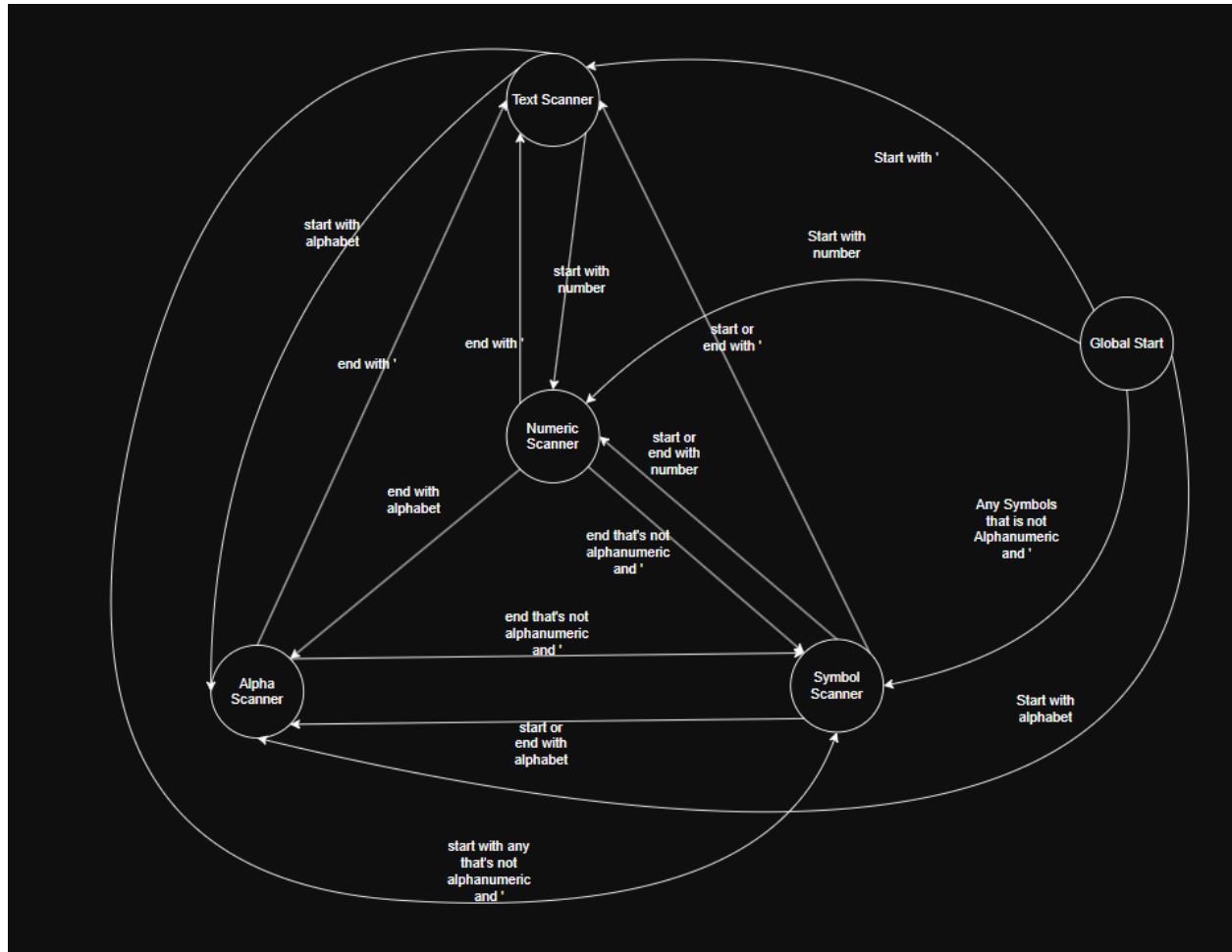






3.2.5. Global DFA

Maksud end adalah karakter terakhir dari scanner sebelumnya, sedangkan start merupakan karakter baru yang diterima setelah keluar dari scanner



3.3. Implementasi DFA

DFA tidak diimplementasikan menggunakan class. Di sini yang menjadi class adalah seluruh lexical analyzer. DFA dibedakan menjadi global DFA dan specific DFA yang didefinisikan sebagai scanners berbeda-beda.

3.3.1. Global DFA

Global DFA ini adalah abstraksi global yang akan menghapus *white space* kemudian akan mengenali jenis input kemudian memanggil scanner spesifik untuk mengenali token tersebut. Global DFA ini berperan sebagai *dispatcher* yang akan memanggil 4 jenis scanner spesifik yaitu

1. Alpha DFA
2. Numeric DFA
3. Text DFA

4. Symbol DFA

Pendekatan implementasi kami gunakan untuk mengelompokan berdasarkan jenis input dan mempermudah pembagian tanggung jawab DFA dan mempermudah implementasi dan pengimplementasian DFA. Dalam class *Lexical Analyzer* global DFA ini dicantumkan dalam method `getNextToken()` dan dijadikan pula menjadi start state untuk memanggil DFA yang lain.

3.3.2. Alpha DFA (scanner Alpha)

3.3.2.1. Overview

Alpha DFA bertugas untuk mengambil token dari input *alphanumeric*. Untuk memperjelas, DFA ini berfungsi membedakan apakah sebuah input itu merupakan *keyword* atau sebuah *ident*. Seperti “var” dengan “varA”, yang mana “var” merupakan *keyword* dan “varA” merupakan *ident*.

Untuk memudahkan memahami semua state DFA Alpha, dibedakan menjadi 3 kategori dari sifat state, dengan kategori pertama adalah state yang masih mampu menjadi *keyword*, state yang sudah pasti *ident* tetapi belum selesai, dan *final state* yang terjadi apabila menemukan karakter non-alfanumerik.

Alpha DFA memiliki penamaan state dengan format spesifik untuk memudahkan navigasi *state*. Format dari penamaan state

a<index keyword pertama pada kamus>_<index keyword kedua pada kamus>..._<index keyword ke-n pada kamus>_i<panjang string yang telah diterima sejauh ini>. Contoh state:

Apabila terdapat kamus *keyword* dengan *keyword* “key” dan “keywarudo” pada index 0 dan 1, maka saat DFA berada di posisi yang sudah menerima string “ke”, dia akan berada di state “a0_1_i2”. Dengan 0 merepresentasikan “key”, 1 merepresentasikan “keywarudo”, dan “i2” merepresentasikan panjang string yang diterima sejauh ini. Panjang string akan ke-reset apabila menemui karakter non-alfanumerik karena itu di luar dari *scope* DFA Alpha.

Dalam kode sebenarnya terdapat 27 *keyword* dalam kamus, namun semua *keyword* diurutkan sesuai alfabet untuk memudahkan matching.

3.3.2.2. Definisi DFA

Berikut adalah deklarasi formal dari alfabet yang digunakan oleh DFA ini.

$\Sigma = \{a \dots z, A \dots Z, 0 \dots 9\}$ dan $\bar{\Sigma}$ merepresentasikan karakter non-alfanumerik

Kemudian himpunan dari *state* adalah:

$$Q = \{q_{start}, q_{\emptyset}, q_{C,l}, q_{F,kw}, q_{F,id}\}$$

Dengan

q_{start} : *State* awal sebelum karakter apa apa dikonsumsi

$q_{\emptyset}$: *State* ketika tidak ada lagi *keyword* di kamus yang cocok dengan string

$q_{C,l}$: *State* yang dibuat secara dinamis seperti yang telah dijelaskan pada overview. Dengan C adalah *subset* dari kamus seluruh *keyword* yang tidak kosong ($C \subseteq \{0, 1, \dots, N-1\}$) yang masih cocok dengan string yang telah dibaca.

$q_{F,kw}$: *Final state* yang menerima *keyword*

$q_{F,id}$: *Final state* yang menerima *identifier*

A. Fungsi transisi untuk $c \in \Sigma$

Kemudian fungsi transisi untuk karakter yang bagian dari karakter alfanumerik ($c \in \Sigma$) didefinisikan sebagai berikut:

$$\delta(q_{start}, c) = \begin{cases} q_{C_{new},1} & \text{if } C_{new} \neq \emptyset \\ q_{\emptyset} & \text{if } C_{new} = \emptyset \end{cases}$$

Yang artinya dalam *start state*, apabila membaca c maka apabila subset baru *keywords* yang cocok (C_{new}) tidaklah kosong, maka *state* berubah menjadi $q_{C_{new},1}$. Contoh dalam kode, saat *start state* maka

State a0 (q_{start})

a => State a0_1_i1

Dengan $C_{new} = \{0, 1\}$ karena dengan diterimanya 'a' keyword yang cocok di kamus hanyalah "and" pada indeks 0 dan "array" pada indeks 1. Terakhir i1 menandakan $l = 1$

Sehingga $\delta(q_{start}, 'a') = q_{\{0,1\},1}$.

$$\delta(q_{C,l}, c) = \begin{cases} q_{C_{new}, l+1} & \text{if } C_{new} \neq \emptyset \\ q_{\emptyset} & \text{if } C_{new} = \emptyset \end{cases}$$

Berikut adalah fungsi transisi dari $q_{C,l}$ dengan C apapun yang tidak kosong. Mirip dengan fungsi transisi sebelumnya, C diganti dengan C_{new} asalkan subset yang baru tidaklah kosong, dan kemudian l bertambah 1 yang merepresentasikan panjang string yang telah dibaca (lexeme).

Terakhir, fungsi transisi untuk alfabet $\Sigma = \{a \dots z, A \dots Z, 0 \dots 9\}$ adalah:

$$\delta(q_{\emptyset}, c) = q_{\emptyset}$$

Terlihat di fungsi-fungsi transisi sebelumnya bahwa $q_{\emptyset}$ dicapai apabila C_{new} menjadi himpunan kosong. Artinya tidak ada lagi keyword yang cocok. Fungsi transisi mengatakan bahwa apabila sudah di *state* ini maka mendapatkan alfabet apapun dari $\Sigma = \{a \dots z, A \dots Z, 0 \dots 9\}$ akan tetap kembali ke $q_{\emptyset}$. Contoh apabila mendapat string “zaza”, maka sudah tidak ada lagi *keyword* pada kamus yang cocok. Sehingga ditambah karakter apapun dari Σ tidak akan membuat subset C_{new} menjadi tidak kosong lagi.

B. Fungsi transisi untuk $c \in \bar{\Sigma}$

$$\delta(q_{\emptyset}, c) \rightarrow q_{F,id}$$

Mengatakan apabila sedang berada di *state* $q_{\emptyset}$ mendapat karakter bagian dari $\bar{\Sigma}$ (karakter non-alfanumerik) maka otomatis akan mencapai *final state identifier*.

$$\delta(q_{C,l}, c) \rightarrow \begin{cases} q_{F,kw} & \text{if } \exists k \in C \text{ where } \text{length}(\text{keyword}_k) = l \\ q_{F,id} & \text{otherwise} \end{cases}$$

Mengatakan apabila sedang berada di $q_{C,l}$ yang mana C bukanlah himpunan kosong, maka apabila mendapat karakter non-alfanumerik akan langsung menuju *final state keyword* apabila dalam himpunan C terdapat indeks yang berkorepondensi ke keyword dengan panjang yang sama dengan l . Apabila tidak ada yang panjangnya sama, contohnya string “arr” akan menyisakan indeks dari *keyword* “array” pada C tetapi “arr” panjangnya tidak cocok, sehingga mencapai *final state identifier*.

Dengan ini DFA alfa $\{Q, \Sigma, q, F, \delta\}$ telah terdefinisi.

3.3.2.3. Implementasi Kode

```
// Looping ini adalah State 1
    while (std::isalnum(static_cast<unsigned
char>(state.peek()))))
    {
        char c = state.advance();
        lexeme += c;

        if (first) {
            filteredKeywords = filterKeywords(keywordsArray,
lexeme);
            first = false;
        } else {
            filteredKeywords =
filterKeywords(filteredKeywords, lexeme);
        }

        // LOGGING STATE TRANSITION
        if(filteredKeywords.size()==0){
            //State (number of keywords) represents the state
in which there is no keywords matching left, thus
            //signifying tis an ident
            //e.g "a27" if the total number of keywords are 27
            std::cout << c << " => State a" <<
std::to_string(keywordsArray.size()) << std::endl;
        }
        else if(filteredKeywords.size()>=2){
            //Means there is at least one pair of [keyword,
idx] but the lexeme scanning isnt yet done
            //a<idx1>_<idx2>_<idx3>...._<idxn>_i<length of
lexeme>

            std::cout << c << " => State a" <<
```

```

getState(filteredKeywords) << "_i"<<
std::to_string(lexeme.length()) <<std::endl;
    }
}

```

Kutipan kode di atas menunjukkan *main loop* dari fungsi alpha scanner. Karakter *c* didapatkan dengan melakukan advance. Fungsi helper adalah fungsi yang memperbarui himpunan *C* pada $q_{C,l}$. Di sini *C* direpresentasikan dengan *filteredKeywords*.

If clause yang pertama merepresentasikan fungsi transisi

$$\delta(q_{C,l}, c) = \begin{cases} q_{C_{new}, l+1} & \text{if } C_{new} \neq \emptyset \\ q_{\emptyset} & \text{if } C_{new} = \emptyset \end{cases}$$

Dan If clause yang kedua (pada else if) merepresentasikan fungsi transisi

$$\delta(q_{\emptyset}, c) = q_{\emptyset}$$

Pada kode, $q_{\emptyset}$ direpresentasikan dengan format `a<panjang kamus seluruh keyword>`.

3.3.3. Numeric DFA (scanner Numeric)

3.3.3.1. Overview

Numeric DFA hanya bertugas membaca angka, DFA ini mengambil input angka 0-9 dan memutuskan apabila hasilnya adalah "intcon", "realcon", ataupun error apabila membaca numeric yang tidak valid. Numeric DFA bekerja hanya dengan melihat pola membaca angka. Karena hanya memiliki sedikit state maka berbeda dengan alpha DFA, numeric DFA memiliki state yang secara eksplisit di definisikan dari state: $n_0 - n_3$.

3.3.3.2. Definisi transisi Numeric DFA

1. Dimulai dari n_0 , input numeric digit pertama akan menjadi transisi ke n_1
2. Dari n_1 , jika membaca digit lagi, tetap di n_1 . Jika membaca '.' (titik yang menyimbolkan koma dalam bilangan real), pindah ke n_2 . Jika membaca non-digit/non-dot, token selesai sebagai intcon

3. Dari n2, wajib ada minimal satu digit setelah ' , jika ada digit, pindah ke n3, jika tidak maka akan menghasilkan error "IncompleteReal" (contoh '6.notSeven')
4. Dari n3, selama membaca numeric maka akan tetap bertahan di n3 sampai membaca non-numeric dimana token akan selesai menjadi realcon.
5. Jika setelah digit muncul e/E akan dianggap tidak valid.

3.3.3.3. Implementasi Kode

```
Token scanText(LexerState &state)
{
    int startLine = state.currentLine;
    int startColumn = state.currentColumn;
    std::string lexeme;

    if (state.advance() != '\\')
    {
        return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn);
    }

    std::cout << "' => State t0\\n";

    // t1 if next '
    if (state.peek() == '\\')
    {
        state.advance();
        std::cout << "' => State t1\\n";

        // t2 second '
        if (state.peek() == '\\')
        {
            state.advance();
            std::cout << "' => State t2\\n";

            // t3: cek quote ketiga
            if (state.peek() == '\\')
```

```

        {
            state.advance();
            std::cout << "'" => State t3\n";

            // t4 4th '
            if (state.peek() == '\'' )
            {
                state.advance();
                std::cout << "'" => State Final State => Gotten:
charcon(')\n";
                return Token(TokenType::CHARCON, "'", startLine,
startColumn);
            }

            std::cout << "=> State Final State => Gotten: error(''
)\n";
            return Token(TokenType::ERROR_TOKEN,
ErrorType::IllegalChar, startLine, startColumn, "''");
        }

        std::cout << "=> State Final State => Gotten: stringcon(" <<
lexeme << ")\n";
        return Token(TokenType::STRINGCON, lexeme, startLine,
startColumn);
    }

    std::cout << "=> State Final State => Gotten: stringcon(" <<
lexeme << ")\n";
    return Token(TokenType::STRINGCON, lexeme, startLine,
startColumn);
}

// t5 char after first '
if (state.isAtEnd())
{
    std::cout << "=> State Final State => Gotten: error(" << lexeme
<< ")\n";
    return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn, lexeme);
}

```

```

    }

    char first = state.advance();
    if (first == '\n' || first == '\0')
    {
        std::cout << "=> State Final State => Gotten: error(" << lexeme
        << ")\n";
        return Token(TokenType::ERROR_TOKEN,
        ErrorType::UnterminatedString, startLine, startColumn, lexeme);
    }

    lexeme += first;
    std::cout << first << " => State t4\n";

    if (state.peek() == '\\')
    {
        state.advance();
        std::cout << "'" => State Final State => Gotten: charcon(" <<
        lexeme << ")\n";
        return Token(TokenType::CHARCON, lexeme, startLine,
        startColumn);
    }

    // t5 string
    std::cout << "=> State t5\n";
    while (!state.isAtEnd())
    {
        char c = state.advance();

        if (c == '\\')
        {
            // t6 meetin ' mid string
            std::cout << "'" => State t6\n";

            if (state.peek() == '\\')
            {
                state.advance();
                lexeme += '\\';
                std::cout << "'" => State t5\n";
            }
        }
    }

```

```

        continue;
    }

    std::cout << "=> State Final State => Gotten: stringcon(" <<
lexeme << ")\n";
    return Token(TokenType::STRINGCON, lexeme, startLine,
startColumn);
}

if (c == '\n' || c == '\0')
{
    std::cout << "=> State Final State => Gotten: error(" <<
lexeme << ")\n";
    return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn, lexeme);
}

lexeme += c;
std::cout << c << " => State t5\n";
}

std::cout << "=> State Final State => Gotten: error(" << lexeme <<
")\n";
return Token(TokenType::ERROR_TOKEN, ErrorType::UnterminatedString,
startLine, startColumn, lexeme);
}

```

3.3.4. Text DFA (scanner Text)

3.3.4.1. Overview

Text DFA bertugas membaca literal yang dimulai dengan tanda ' untuk membedakan token 'stringcon', 'charcon', atau error apabila tidak ada state yang memenuhi kondisi string. Dalam bahasa arion tanda petik ganda menandakan *escape character* dalam mewakili karakter ' literal. Contoh, dalam input 'Baca Buku "KBBI"' Memberikan output stringcon ('Baca Buku 'KBBI'). Text DFA memiliki state t0 - t6.

3.3.4.2. Definisi transisi Text DFA

1. Dimulai dari t0, input ' pertama akan menjadi transisi ke t1
2. Dari t1, scanner sudah membaca opening ' pertama. Jika karakter setelah opening ' bukan ' , karakter itu dibaca sebagai character pertama isi literal dan scanner transisi ke t5, jika karakter setelah opening ' adalah ' , scanner transisi t2 sekaligus menghasilkan token `stringcon("")` kosong.
3. Dari t2, jika ' kedua membaca diikuti ' , scanner pindah ke t3
4. Dari t3, jika ' ketiga tidak membaca ' lagi, pola "" dianggap tidak valid dan menghasilkan error, sedangkan jika ' ketiga masih diikuti ' , scanner pindah ke t4.
5. Dari t4, jika ' keempat ada, pola "" diterima sebagai charcon yang berisi `charcon(')` literal.
6. Dari t5, jika setelah character literal pertama lalu membaca ' maka token akan selesai sebagai charcon satu character. Jika membaca character lain maka akan transisi ke t6
7. Dari t6, jika membaca character apapun akan bertahan di t6, dan apabila membaca ' maka akan menghasilkan token `stringcon` dengan semua character yang telah dibaca sebagai isinya dan transisi ke t2
8. Dari t2, scanner menentukan hasil ' di tengah string. Jika membaca ' lagi maka ' tersebut akan dianggap sebagai *escape character* untuk literal ' dan transisi ke t6 untuk membaca string lebih lanjut hingga menemukan ' sebagai penutup string.

3.3.4.3. Implementasi Kode

```
Token scanText(LexerState &state)
{
    int startLine = state.currentLine;
    int startColumn = state.currentColumn;
    std::string lexeme;

    if (state.advance() != '\\')
    {
        return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn);
    }
}
```



```

// ' ' -> empty string
// ''' -> invalid
// '''' -> charcon(')
if (state.peek() == '\\')
{
    state.advance();

    if (state.peek() == '\\')
    {
        state.advance();

        if (state.peek() == '\\')
        {
            state.advance();
            return Token(TokenType::CHARCON, "'", startLine,
startColumn);
        }

        return Token(TokenType::ERROR_TOKEN, ErrorType::IllegalChar,
startLine, startColumn, "''");
    }

    return Token(TokenType::STRINGCON, lexeme, startLine,
startColumn);
}

// State t0 reads char
if (state.isAtEnd())
{
    return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn, lexeme);
}

char first = state.advance();

if (first == '\\n' || first == '\\0')
{
    return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn, lexeme);
}

```

```

    }

    lexeme += first;

    // State t1: if next thing is ' then charcon, else stringcon
    if (state.peek() == '\\')
    {
        state.advance();
        return Token(TokenType::CHARCON, lexeme, startLine,
startColumn);
    }

    // State t2: stringcon
    while (!state.isAtEnd())
    {
        char c = state.advance();

        if (c == '\\')
        {
            if (state.peek() == '\\')
            {
                state.advance();
                lexeme += '\\';
                continue;
            }

            return Token(TokenType::STRINGCON, lexeme, startLine,
startColumn);
        }

        if (c == '\\n' || c == '\\0')
        {
            return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedString, startLine, startColumn, lexeme);
        }

        lexeme += c;
    }

```

```
        return Token(TokenType::ERROR_TOKEN, ErrorType::UnterminatedString,
startLine, startColumn, lexeme);
    }
```

3.3.5. Symbol DFA (scanner Symbol)

3.3.5.1. Overview

DFA Symbol bertugas untuk mengambil token dari input yang berupa simbol (operator matematika, tanda baca, tanda kurung) serta menangani blok komentar, baik menggunakan format (*...*) maupun {...}.

Untuk simbol yang terdefinisi pasti tunggal seperti semicolon (;), comma (,) dan plus (+). Kode program akan langsung mengembalikan tipe token yang sesuai dengan simbol.

Untuk simbol yang terdefinisi maksimal dua karakter seperti simbol less (<) dapat bertipe token less equal (<=), not equal (<>), ataupun tetap less (<) tergantung input simbol selanjutnya.

Komentar dalam bahasa arion memiliki struktur (*...*) dan {} sehingga jika program menemukan simbol '{' dan '(' akan memulai loop yang akan berakhir jika menemukan terminator. Khusus '{' karena dapat memiliki dua token yaitu left parenthesis dan comment. Program akan mengecek input setelah '{' jika '*' akan berubah menjadi comment, jika bukan '*' akan mengembalikan tipe left parenthesis.

3.3.5.2. Definisi DFA

Definisi state symbol dapat didefinisikan sebagai

$$M = (Q, \Sigma, \delta, q_0, F)$$

Dalam hal ini

- Q : $s_1, \dots, s_{25}$ (jumlah state finite)
- Σ : ASCII (huruf, angka, dan simbol)
- δ : $s_1, s_2, s_3, s_4, s_5, s_6$ (state perantara)
- q_0 : state awal
- F : $s_7, \dots, s_{25}$ (final state)

3.3.5.3. Implementasi Kode

```
Token scanSymbol(LexerState &state)
{
    int startLine = state.currentLine;
    int startColumn = state.currentColumn;
    char c = state.advance();
    std::cout << c << " => State s0\n";

    switch (c)
    {
    case ';':
        std::cout << "=> State s7 => Gotten: semicolon(;)\\n";
        return Token(TokenType::SEMICOLON, ";", startLine, startColumn);
    case ',':
        std::cout << "=> State s8 => Gotten: comma(,)\\n";
        return Token(TokenType::COMMA, ",", startLine, startColumn);
    case '.':
        std::cout << "=> State s9 => Gotten: period(.)\\n";
        return Token(TokenType::PERIOD, ".", startLine, startColumn);
    case '+':
        std::cout << "=> State s10 => Gotten: plus(+)\\n";
        return Token(TokenType::PLUS, "+", startLine, startColumn);
    case '-':
        std::cout << "=> State s11 => Gotten: minus(-)\\n";
        return Token(TokenType::MINUS, "-", startLine, startColumn);
    case '*':
        std::cout << "=> State s12 => Gotten: times(*)\\n";
        return Token(TokenType::TIMES, "*", startLine, startColumn);
    case '/':
        std::cout << "=> State s13 => Gotten: rdiv(/)\\n";
        return Token(TokenType::RDIV, "/", startLine, startColumn);

    // comment v1
    case '(':
        if (state.peek() == '*')
        {
            state.advance(); // consume '*'
        }
    }
```

```

        std::cout << "(* => State s5\n";
        std::string commentText;
        while (!state.isAtEnd())
        {
            char cc = state.advance();
            if (cc == '*' && state.peek() == ')')
            {
                state.advance(); // consume ')'
                std::cout << "*) => State s26\n";
                std::cout << "=> State s26 => Gotten: comment\n";
                return Token(TokenType::COMMENT, commentText,
startLine, startColumn);
            }
            commentText += cc;
        }
        std::cout << "=> State s27 => Gotten: error(comment)\n";
        return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedComment, startLine, startColumn);
    }

    std::cout << "=> State s14 => Gotten: lparent()\n";
    return Token(TokenType::LPARENT, "(", startLine, startColumn);

case ')':
    std::cout << "=> State s15 => Gotten: rparent()\n";
    return Token(TokenType::RPARENT, ")", startLine, startColumn);
case '[':
    std::cout << "=> State s16 => Gotten: lbrack([\n";
    return Token(TokenType::LBRACK, "[", startLine, startColumn);
case ']':
    std::cout << "=> State s17 => Gotten: rbrack(]\n";
    return Token(TokenType::RBRACK, "]", startLine, startColumn);

case ':':
    if (state.peek() == '=')
    {
        state.advance();
        std::cout << "= => State s1\n";
        std::cout << "=> State s18 => Gotten: becomes(=)\n";
        return Token(TokenType::BECOMES, ":", startLine,

```

```

startColumn);
    }

    std::cout << "=> State s19 => Gotten: colon(:)\n";
    return Token(TokenType::COLON, ":", startLine, startColumn);

case '<':
    if (state.peek() == '=')
    {
        state.advance();
        std::cout << "= => State s2\n";
        std::cout << "=> State s20 => Gotten: leq(<=)\n";
        return Token(TokenType::LEQ, "<=", startLine, startColumn);
    }
    if (state.peek() == '>')
    {
        state.advance();
        std::cout << "> => State s3\n";
        std::cout << "=> State s21 => Gotten: neq(<>)\n";
        return Token(TokenType::NEQ, "<>", startLine, startColumn);
    }
    std::cout << "=> State s22 => Gotten: lss(<)\n";
    return Token(TokenType::LSS, "<", startLine, startColumn);

case '>':
    if (state.peek() == '=')
    {
        state.advance();
        std::cout << "= => State s4\n";
        std::cout << "=> State s23 => Gotten: geq(>=)\n";
        return Token(TokenType::GEQ, ">=", startLine, startColumn);
    }
    std::cout << "=> State s24 => Gotten: gtr(>)\n";
    return Token(TokenType::GTR, ">", startLine, startColumn);

case '=':
    if (state.peek() == '=') {
        state.advance();
        std::cout << "= => State s25\n";
        std::cout << "=> State s25 => Gotten: eql(==)\n";
    }

```

```

        return Token(TokenType::EQL, "==", startLine, startColumn);
    }

    std::cout << "=> State s29 => Gotten: error(=)\n";
    return Token(TokenType::ERROR_TOKEN, ErrorType::IllegalChar,
startLine, startColumn, "=");
    // comment v2
    case '{':
    {
        std::cout << "{ => State s6\n";
        std::string commentText;
        while (!state.isAtEnd() && state.peek() != '}')
        {
            commentText += state.advance();
        }
        if (state.isAtEnd())
        {
            std::cout << "=> State s27 => Gotten: error(comment)\n";
            return Token(TokenType::ERROR_TOKEN,
ErrorType::UnterminatedComment, startLine, startColumn);
        }
        state.advance(); // consume '}'
        std::cout << "} => State s26\n";
        std::cout << "=> State s26 => Gotten: comment\n";
        return Token(TokenType::COMMENT, commentText, startLine,
startColumn);
    }

    default:
        std::cout << "=> State s28 => Gotten: error(" << c << ")\n";
        return Token(TokenType::ERROR_TOKEN, ErrorType::IllegalChar,
startLine, startColumn, std::string(1, c));
    }
}

```

3.4. Alur Program

Program pada *main* dimulai dengan memasukkan path dari source code arion yang ingin diproses. *Input* pada *main* masih dilanjut dengan memasukkan path dari output yang diinginkan. Setelah mem-*passing* data dari source code yang diberikan pada *object LexicalAnalyzer*, langsung dilakukan *TokenizeAll()* yang merupakan *method* pada class *LexicalAnalyzer*. Dengan pemanggilan *method* ini main loop dari program Lexical Analysis ini berjalan.

3.4.1. Inisialisasi *LexicalAnalyzer*

Sebelum memulai, objek *LexicalAnalyzer* membersihkan *errorTokens* untuk memastikan kondisi mulai yang bersih. Kemudian dibuatkan sebuah array (menggunakan *vector* dari C++) kosong untuk menampung token, Dan inisialisasi pada *TokenizeAll* terakhir dilakukan dengan mengambil token pertama menggunakan *getNextToken()*

3.4.2. *While Loop* Inti

Loop ini diatur oleh satu kondisi utama yaitu "*while (currentToken.type != TokenType::END_OF_FILE)*". Loop akan terus berputar selama lexer belum mencapai bagian paling akhir dari string kode sumber.

Jika tipe tokennya adalah *ERROR_TOKEN* (seperti string yang tidak ditutup atau karakter ilegal), token tersebut dimasukkan ke dalam daftar terpisah bernama *errorTokens*. Jika tokennya valid, token tersebut dimasukkan ke dalam daftar utama tokens.

Di akhir badan loop, program memanggil "*currentToken = getNextToken()*". Ini meminta lexer untuk membaca potongan kode sumber berikutnya dan menyiapkannya untuk iterasi loop selanjutnya.

3.4.3. Pemaju Token

Di balik layar, setiap kali loop memanggil *getNextToken()*, lexer melakukan serangkaian operasi untuk menentukan token apa selanjutnya:

- Pertama, ia memanggil *skipWhitespace()* untuk memakan semua spasi, tab, atau baris baru (newline).

- Jika tidak ada teks yang tersisa, ia mengembalikan token `END_OF_FILE`, yang akan memicu loop utama untuk berhenti pada pengecekan berikutnya.
- Jika masih ada teks yang tersisa, maka program mengintip (peek) karakter pertama dan menggunakan rantai if-else untuk mendistribusikan tugas ke scanner spesifik kamu:
 - Jika berupa huruf atau `_`, ia mengarahkannya ke `scanAlpha` (untuk Keyword dan Identifier).
 - Jika berupa angka, dia mengarahkannya ke `scanNumeric` (untuk Int dan Real).
 - Jika berupa `'`, dia mengarahkannya ke `scanText` (untuk Char dan String).
 - Selain itu, ia mengarahkannya ke `scanSymbol` (untuk Operator, Tanda Kurung, dan Komentar).

3.4.4. Terminasi

Setelah `getNextToken()` akhirnya mencapai akhir file dan mengembalikan token `END_OF_FILE`, kondisi loop while akan bernilai salah (*false*) dan loop pun berhenti. Program kemudian memasukkan token EOF terakhir itu ke dalam daftar token utama dan mengembalikan vektor yang sudah terisi penuh tersebut ke bagian kompiler manapun yang memintanya.

4. Pengujian

4.1. Pengujian hasil Lexical Analyzer

No	Token/Perma salahan yang dites	input	Hasil Program	Ekspektasi dapat mengenali	Tujuan
1.	programsy, ident, semicolon, varsy, colon, comma, beginsy, becomes, intcon, plus, endsy, period	program Hello; var a, b: Integer; begin a := 5; b := a + 10; end.	programsy ident (Hello) semicolon varsy ident (a) comma ident (b) colon ident (Integer) semicolon beginsy ident (a) becomes intcon (5) semicolon ident (b) becomes ident (a) plus intcon (10) semicolon endsy period endoffile	program -> programsy Hello -> Ident (Hello) ; -> semicolon var -> varsy , -> comma begin -> beginsy a b -> ident := -> becomes + -> plus 5 dan 10 -> intcon end -> endsy . -> period	Struktur Program Dasar & Variabel
2.	constsy, typesy, realcon, charcon, string,	program TestConst; const MaxVal := 100; Pi := 3.14; Grade := 'A'; School := 'IRK'; var x: Integer; begin x := MaxVal; end.	programsy ident (TestConst) semicolon constsy ident (MaxVal) becomes intcon (100) semicolon ident (Pi) becomes realcon (3.14) semicolon	const -> consty 3.14 -> realcon(3.14) 'A' -> charcon('A') 'IRK' -> string('IRK')	Konstanta & Tipe Data

			ident (Grade) becomes charcon (A) semicolon ident (School) becomes string ('IRK') semicolon varsy ident (x) colon ident (Integer) semicolon beginsy ident (x) becomes ident (MaxVal) semicolon endsy period endoffile		
3.	plus, minus, times, idiv, rdiv, imod, intcon, becomes	program TestArith; var a, b, c: Integer; r: Real; begin a := 10 + 3; b := 10 - 3; c := 10 * 3; a := 10 div 3; r := 10 / 3; b := 10 MOD 3; end.	programsy ident (TestArith) semicolon varsy ident (a) comma ident (b) comma ident (c) colon ident (Integer) semicolon ident (r) colon ident (Real) semicolon beginsy ident (a) becomes intcon (10) plus intcon (3) semicolon ident (b) becomes intcon (10) minus intcon (3) semicolon	a b c r -> ident Integer -> ident(Integer) Real -> ident(real) := -> becomes 3 10 -> intcon + -> plus - -> minus * -> times div -> idiv / -> rdiv mod -> imod	Operator Aritmatika Lengkap

			ident (c) becomes intcon (10) times intcon (3) semicolon ident (a) becomes intcon (10) idiv intcon (3) semicolon ident (r) becomes intcon (10) rdiv intcon (3) semicolon ident (b) becomes intcon (10) imod intcon (3) semicolon endsy period endoffile		
4.	notsy, andsy, orsy, eql, neq, gtr, geq, lss, leq	program TestLogic; var a, b: Integer; p, q, r: Boolean; begin p := a == b; q := a <> b; r := a > b; p := a >= b; q := a < b; r := a <= b; p := NOT q; r := p AND q; p := q OR r; end.	programsy ident (TestLogic) semicolon varsy ident (a) comma ident (b) colon ident (Integer) semicolon ident (p) comma ident (q) comma ident (r) colon ident (Boolean) semicolon beginsy ident (p) becomes ident (a)	NOT -> notsy AND -> endy OR -> orsy == -> eql <> -> nql > -> gtr >= -> geq < -> lss <= -> leq	Operator Logika & Perbandin gan

			eql ident (b) semicolon ident (q) becomes ident (a) neq ident (b) semicolon ident (r) becomes ident (a) gtr ident (b) semicolon ident (p) becomes ident (a) geq ident (b) semicolon ident (q) becomes ident (a) lss ident (b) semicolon ident (r) becomes ident (a) leq ident (b) semicolon ident (p) becomes notsy ident (q) semicolon ident (r) becomes ident (p) andsy ident (q) semicolon ident (p) becomes ident (q) orsy ident (r) semicolon endsy period endoffile		
--	--	--	---	--	--

5.	ifsy, theny, elsesy, lparent, rparent	<pre> program TestIf; var x, y: Integer; begin x := 10; if (x > 5) then y := 1 else y := 0; end. </pre>	<pre> programsy ident (TestIf) semicolon varsy ident (x) comma ident (y) colon ident (Integer) semicolon beginsy ident (x) becomes intcon (10) semicolon ifsy lparent ident (x) gtr intcon (5) rparent theny ident (y) becomes intcon (1) error ident (y) becomes intcon (0) semicolon endsy period endoffile </pre>	<pre> if -> ifsy then -> theny else -> elsesy (-> lparent) -> rparent </pre>	Struktur If-Then-Else
6.	whilesy, dosy	<pre> program TestWhile; var i: Integer; begin i := 1; while (i <= 10) do begin i := i + 1; end; end. </pre>	<pre> programsy ident (TestWhile) semicolon varsy ident (i) colon ident (Integer) semicolon beginsy ident (i) becomes intcon (1) </pre>	<pre> while -> whilesy do -> dosy </pre>	Struktur While-Do

			semicolon whilesy lparent ident (i) leq intcon (10) rparent dosy beginsy ident (i) becomes ident (i) plus intcon (1) semicolon endsy semicolon endsy period endoffile		
7.	forsy, tosy, downtosy	program TestFor; var i: Integer; begin for i := 1 to 5 do i := i + 1; for i := 5 downto 1 do i := i - 1; end.	programsy ident (TestFor) semicolon varsy ident (i) colon ident (Integer) semicolon beginsy forsy ident (i) becomes intcon (1) tosy intcon (5) dosy ident (i) becomes ident (i) plus intcon (1) semicolon forsy ident (i) becomes intcon (5) downtosy intcon (1) dosy ident (i)	for -> forsy to -> tosy down -> downsy	Struktur For-To & For-Downto

			becomes ident (i) minus intcon (1) semicolon endsy period endoffile		
8.	repeatsy, untilsy	<pre> program TestRepeat; var i: Integer; begin i := 0; repeat i := i + 1; until (i >= 10); end.</pre>	<pre> programsy ident (TestRepeat) semicolon varsy ident (i) colon ident (Integer) semicolon beginsy ident (i) becomes intcon (0) semicolon repeatsy ident (i) becomes ident (i) plus intcon (1) semicolon utilsy lparent ident (i) geq intcon (10) rparent semicolon endsy period endoffile</pre>	reapat -> repeatsy until -> untilsy	Struktur REPEAT-UNTIL
9.	casesy, ofsy	<pre> program TestCase; var day: Integer; begin day := 1; case day of 1: day := 2; 2: day := 3;</pre>	<pre> programsy ident (TestCase) semicolon varsy ident (day) colon ident (Integer) semicolon</pre>	case -> casesy of -> ofsy	Struktur CASE-OF

		end; end.	beginsy ident (day) becomes intcon (1) semicolon casesy ident (day) ofsy intcon (1) colon ident (day) becomes intcon (2) semicolon intcon (2) colon ident (day) becomes intcon (3) semicolon endsy semicolon endsy period endoffile		
10.	Arraysy, lbrack, rbrack	program TestArray; var arr: array [1..5] of Integer; i: Integer; begin arr[1] := 99; i := arr[1]; end.	programsy ident (TestArray) semicolon varsy ident (arr) colon arraysy lbrack intcon (1) period period intcon (5) error ofsy ident (Integer) semicolon ident (i) colon ident (Integer) semicolon beginsy ident (arr) lbrack intcon (1) error	array -> arraysysy [-> lbrack] -> rbrack	Array

			becomes intcon (99) semicolon ident (i) becomes ident (arr) lbrack intcon (1) error semicolon endsy period endoffile		
11.	recordsy, typesy,	program TestRecord; type Point := record x: Integer; y: Integer; end; var p: Point; begin p.x := 3; p.y := 4; end.	programsy ident (TestRecord) semicolon typesy ident (Point) becomes record ident (x) colon ident (Integer) semicolon ident (y) colon ident (Integer) semicolon endsy semicolon varsy ident (p) colon ident (Point) semicolon beginsy ident (p) period ident (x) becomes intcon (3) semicolon ident (p) period ident (y) becomes intcon (4) semicolon endsy period	record -> recordsy type-> typesy	Record & type

			endoffile		
12.	functions, proceduresy	<pre> program TestSubprogram; function Add(a, b: Integer): Integer; begin Add := a + b; end; procedure Print(x: Integer); begin x := x + 1; end; var result: Integer; begin result := Add(3, 4); Print(result); end. </pre>	<pre> programsy ident (TestSubprogra m) semicolon functionsy ident (Add) lparent ident (a) comma ident (b) colon ident (Integer) rparent colon ident (Integer) semicolon beginsy ident (Add) becomes ident (a) plus ident (b) semicolon endsy semicolon proceduresy ident (Print) lparent ident (x) colon ident (Integer) rparent semicolon beginsy ident (x) becomes ident (x) plus intcon (1) semicolon endsy semicolon varsy ident (result) colon ident (Integer) semicolon </pre>	<pre> function -> functionsy procedure -> proceduresy </pre>	Function & Procedure

			beginsy ident (result) becomes ident (Add) lparent intcon (3) comma intcon (4) rparent semicolon ident (Print) lparent ident (result) rparent semicolon endsy period endoffile		
13.	comment	<pre> program TestComment; { Ini adalah komentar blok } var x: Integer; { komentar inline } begin (* komentar gaya lain *) x := 42; end.</pre>	<pre> programsy ident (TestComment) semicolon comment (' Ini adalah komentar blok ') varsy ident (x) colon ident (Integer) semicolon comment (' komentar inline ') beginsy comment (' komentar gaya lain ') ident (x) becomes intcon (42) semicolon endsy period endoffile</pre>	<pre> { Ini adalah komentar blok } -> comment (' Ini adalah komentar blok ') { komentar inline } -> (' komentar inline ') (* komentar gaya lain *) -> comment (' komentar gaya lain ') </pre>	Comment
14.	Verifikasi bahwa keyword dan	<pre> PROGRAM CaseTest; VAR myVar: INTEGER;</pre>	<pre> programsy ident (CaseTest) semicolon</pre>	PROGRAM -> programsy	Case-Inse nsitive Identifier

	ident bersifat case-insensitive	<pre> BEGIN MyVar := 7; IF (myvar > 0) THEN myVAR := 0; END.</pre>	<pre> varsy ident (myVar) colon ident (INTEGER) semicolon beginsy ident (MyVar) becomes intcon (7) semicolon ifsy lparent ident (myvar) gtr intcon (0) rparent thensy ident (myVAR) becomes intcon (0) semicolon endsy period endoffile</pre>	<pre> VAR -> varsy INTEGER -> ident(INTEGER) BEGIN -> Beginsy MyVar -> ident(MyVar) myvar -> ident(myvar) myVAR -> ident(myVAR) IF -> ifsy THEN -> thensy END -> endsy</pre>	& Keyword
15.	Error Handdling	<pre> program error; var a, b: integer; begin a := 203.; b := @123; writeln('hello'); (* ga lengkap end.</pre>	<pre> programsy ident (error) semicolon varsy ident (a) comma ident (b) colon ident (integer) semicolon beginsy ident (a) becomes semicolon ident (b) becomes intcon (123) semicolon ident (writeln) lparent endoffile tambahan output Lexical errors found:</pre>	Tidak mengenali 203. Tidak mengenali @123 sebagai intcont Tidak mengenali 'hello sebagai string Tidak mengenali (* ga lengkap sebagai comment	

			- Line 7, Col 8: incomplete real: 203. - Line 8, Col 8: illegal character: @ - Line 9, Col 11: unterminated string: hello); - Line 10, Col 3: unterminated comment		
--	--	--	---	--	--

4.2. Pengecekan Implementasi Sesuai DFA

No	Input	State yang Diambil
1	<pre> program Hello; var a, b: Integer; begin a := 5; b := a + 10; end. </pre>	State 0 => p => State a17_18_i1 r => State a17_18_i2 o => State a17_18_i3 g => State a18_i4 r => State a18_i5 a => State a18_i6 m => State a18_i7 => State Final State => Gotten: keyword(program) State 0 => H => State a27 e => State a27 l => State a27 l => State a27 o => State a27 => State Final State => Gotten: ident(Hello) State 0 => ; => State s0 => State s7 => Gotten: semicolon(; ; => char => State 0 => Gotten: semicolon(; State 0 => v => State a25_i1 a => State a25_i2 r => State a25_i3 => State Final State => Gotten: keyword(var) State 0 => a => State a0_1_i1 => State Final State => Gotten: ident(a) State 0 => , => State s0

		=> State s8 => Gotten: comma(,) , => char => State 0 => Gotten: comma(,) State 0 => b => State a2_i1 => State Final State => Gotten: ident(b) State 0 => : => State s0 => State s19 => Gotten: colon(:) : => char => State 0 => Gotten: colon(:) State 0 => l => State a12_i1 n => State a27 t => State a27 e => State a27 g => State a27 e => State a27 r => State a27 => State Final State => Gotten: ident(Integer) State 0 => ; => State s0 => State s7 => Gotten: semicolon(;) ; => char => State 0 => Gotten: semicolon(;) State 0 => b => State a2_i1 e => State a2_i2 g => State a2_i3 i => State a2_i4 n => State a2_i5 => State Final State => Gotten: keyword(begin) State 0 => a => State a0_1_i1 => State Final State => Gotten: ident(a) State 0 => : => State s0 = => State s1 => State s18 => Gotten: becomes(:=) State 0 => 5 => State n1 => State Final State => Gotten: intcon(5) State 0 => ; => State s0 => State s7 => Gotten: semicolon(;) ; => char => State 0 => Gotten: semicolon(;) State 0 => b => State a2_i1 => State Final State => Gotten: ident(b) State 0 => : => State s0 = => State s1 => State s18 => Gotten: becomes(:=) State 0 => a => State a0_1_i1 => State Final State => Gotten: ident(a) State 0 => + => State s0 => State s10 => Gotten: plus(+) + => char => State 0 => Gotten: plus(+)
--	--	---

		State 0 => 1 => State n1 0 => State n1 => State Final State => Gotten: intcon(10) State 0 => ; => State s0 => State s7 => Gotten: semicolon(; ; => char => State 0 => Gotten: semicolon(; State 0 => e => State a8_9_i1 n => State a9_i2 d => State a9_i3 => State Final State => Gotten: keyword(end) State 0 => . => State s0 => State s9 => Gotten: period(. . => char => State 0 => Gotten: period(.)
--	--	---

5. Penutup

5.1. Kesimpulan

Pada Milestone 1 Tugas Besar IF2224 Teori Bahasa Formal dan Otomata, kami telah berhasil mengembangkan sebuah lexical analyzer (lexer) untuk bahasa pemrograman Arion dengan memanfaatkan pendekatan DFA yang diimplementasikan menggunakan bahasa C++. Proses perancangan ini mencakup pembagian aturan DFA ke dalam beberapa kelas pemindai spesifik seperti Specific Scanners untuk mengidentifikasi token secara terstruktur, meliputi token alfanumerik, numerik, teks, serta simbol dan komentar. Selain itu, diagram transisi DFA juga telah disusun untuk memvisualisasikan logika perpindahan state dalam DFA.

Hasil implementasi menunjukkan bahwa program lexer mampu mengonversi source code menjadi himpunan token secara akurat melalui pemindaian karakter demi karakter yang dikendalikan oleh LexerState. Selain itu, implementasi juga mengimplementasikan mekanisme error handling dengan program akan mengisolasi menjadi Error Token dan tetap melanjutkan proses pemindaian hingga akhir file. Pengujian program juga telah dilakukan menggunakan setidaknya lima kasus uji yang berbeda dengan seluruh hasil eksekusi, baik input maupun output yang telah didokumentasikan secara lengkap dalam bentuk tangkapan layar.

5.2. Pesan

- Billie Bhaskara Wibawa - 13524024

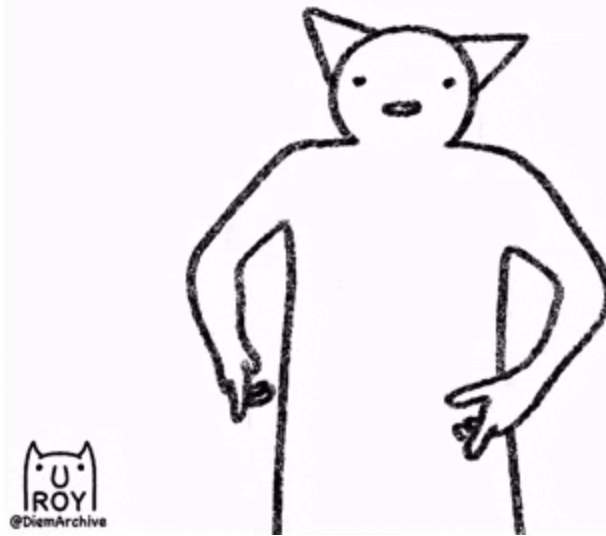
Mungkin dari spek lebih banyak pembahasan edge case (yang ga terlalu niche). Like right now banyak banget pembahasan di QNA yang harusnya juga dari awal ada di spek. But its okay, still a fun tubes (masih baru 1 milestone jir).

- Raysha Erviandika Putra - 13524050

Di ruang lab yang dingin setelah aku dengan gembira mengerjakan tubes TBFO, aku lihat Hoshimachi Suisei berdiri di depan papan tulis sambil mengetuk spidol pelan dari jauh, dia pun marah ketika melihatku mendeklarasikan "" sebagai char?! yang valid, "hmmpphh"

- Muhammad Naufal Romi Annafi - 13524058

Let it ride



"Hmm, large celery" - greedy rabbit

- Dzaki Ahmad Al Hussainy - 13524084

Khusus untuk leker ini memang harus dibuat bagus sih supaya gampang di milestone berikutnya, kak review dong pekerjaan kami :D

6. Lampiran

- Repository Program

Github: <https://github.com/naufalromi/JEE-Tubes-IF2224-2026>

- Diagram DFA

Draw.io:

<https://drive.google.com/file/d/1wsII5sRsc1Em2vMyM58faliV78-yWXhn/view?usp=sharing>

- Tabel Pembagian Tugas

Nama	NIM	Workload	Persentase
Billie Bhaskara Wibawa	13524024	Implementasi : Alpha scanner, State maker DFA Diagram : Alpha scanner, laporan Memandu : Implementasi State DFA	25%
Raysha Erviandika Putra	13524050	Implementasi : Error trace, Numeric scanner, text scanner, symbol scanner Diagram : Alpha scanner	25%
Muhammad Naufal Romi Annafi	13524058	Diagram : Global DFA, Numeric Scanner, text scanner, Symbol scanner Laporan	25%
Dzaki Ahmad Al Hussainy	13524084	PrePlanning : Pembuatan Arsitektur Lexer	25%

		Testing : implementasi Struktur Program awal, Rough Alpha Scanner	
--	--	---	--